OpenGeoSys from zero to a publishable study

Hydro-mechanical screening of cyclic hydrogen storage in a depleted, microporous chert gas reservoir near critically stressed Midcontinent faults

An "Aetna-style" generic screening model (Barber County, Kansas analogue)

Installation · OGS set-up · 30-day workflow · starter models · verified references

Software	OpenGeoSys 6.5.9 and ogstools 0.8.2 (pip), Python 3.12/3.13, gmsh 4.15, ParaView
Platform	Windows 10 (build 19041 or later) or Windows 11 + WSL2 Ubuntu + uv + Python 3.12 (primary); native Windows pip and Docker as fallbacks
Companion files	Repository abrarmahir31/h2-chert-storage-ogs, folder starter/ (all scripts were run)
Status tags	**[Tested by Claude on Linux]** = run by Claude in a Linux sandbox. **[Not tested on Windows]** = written from official docs. **TO-VERIFY** = not yet verified. **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]** = run by the user on the lab PC.
Version	1.0, 12 November 2026

WARNING

This is a generic screening study built from cited literature values and clearly labelled synthetic values. It is not a site-specific prediction for the Aetna Gas Area. Every model property carries a source tag in starter/properties/properties.csv.

Contents

0. How to use this guide 4
1. Feasibility and honesty 5
1.1 Working title options 5
1.2 Q1: Is dual porosity / dual permeability available for two-phase flow? 5
1.3 Q2: Is there relative-permeability or capillary hysteresis? 5
1.4 Q3: Are hydrogen properties built in? 6
1.5 Q4: How can a fault be represented, and how is Coulomb stress computed? 7
1.6 Q5: What is already published, and what is new? 8
1.7 The three limits (repeat these in the manuscript) 8
1.8 Compute plan for your PC 9
1.9 What was verified, and how 10
2. Installing OGS on Windows 10 or 11 11
2.1 Check your PC 11
2.2 Route A: WSL2 + Ubuntu + uv + Python 3.12 (primary, verified on Windows) 11
2.3 VS Code and ParaView 13
2.4 Route B: native Windows pip (not tested) 13
2.5 Route C: Docker container (not tested on Windows) 14
2.6 Verify the installation with real runs 14
2.7 Updating OGS 14
2.8 Most common errors and how to fix them 15
3. Setting up OGS for this study 17
3.1 Project folder layout 17
3.2 How a project file (.prj) is organised 17
3.3 Making meshes 19
3.4 Running a simulation 19
3.5 Post-processing 19
3.6 Choosing the process 20
3.7 The starter models 20
4. The 30-day plan 23
Day 1 – Check your PC and switch on WSL2 24
Day 2 – Install Python 3.12 (uv), OGS, VS Code and ParaView 26
Day 3 – Run the official benchmarks 29
Day 4 – Single-phase flow: the Theis well test 31
Day 5 – Mechanics: stress around a borehole (Kirsch) 33
Day 6 – Hydro-mechanics: effective stress and consolidation 35
Day 7 – Meshing with gmsh and ogstools (Milestone 1) 37
Day 8 – Time stepping, convergence and the log file 39
Day 9 – Two-phase flow theory and the McWhorter benchmark 41
Day 10 – Verification note 1: McWhorter in your own words 43
Day 11 – Two-phase flow with mechanics: Liakopoulos 45
Day 12 – Hydrogen and gas appearance: Bourgeat benchmark 47
Day 13 – Property block 1: hydrogen and brine 49
Day 14 – Property block 2: chat petrophysics (Milestone 2) 51
Day 15 – Property block 3: capillary, relative permeability and trapping 53
Day 16 – Property block 4: rock mechanics and stress state 55

Day 17 – Build the radial (well) model....57
Day 18 – Radial model: full cycles and both trapping brackets....59
Day 19 – Build the fault cross-section....61
Day 20 – Coulomb stress on the fault....63
Day 21 – Experiment design and cushion-gas study (Milestone 3)....65
Day 22 – Trapping bracket sensitivity....67
Day 23 – Sensitivity: permeability, micro-porosity proxy, capillarity, rate....69
Day 24 – Fault sensitivity and the safe operating window....71
Day 25 – Mesh and time-step convergence....73
Day 26 – Analysis and publication-quality figures....75
Day 27 – Results tables and the methods section....77
Day 28 – Manuscript outline, introduction and limitations (Milestone 4)....78
Day 29 – Reproducibility package....80
Day 30 – Self-review and next steps....82
5. Risk register....84
6. Manuscript skeleton....85
6.1 Outline....85
6.2 Methods template....85
6.3 Limitations (copy into the paper)....85
6.4 Reproducibility package....86
6.5 Self-review checklist for a Q1 journal....86
Appendix A. Glossary....87
Appendix B. Cheat sheet (verified)....88
Appendix C. Troubleshooting guide....89
Appendix D. References....90
Appendix E. Verification log (what Claude ran)....92
Appendix F. Model properties and their sources....93

0. How to use this guide

This guide takes you from no OpenGeoSys (OGS) experience to a first manuscript draft in 30 days of 6 hours (180 hours). Read sections 1-3 once (they are your reference), then work through the day pages in section 4. Sections 5-6 and the appendices are for looking things up.

Marker	Meaning
[Tested by Claude on Linux]	Claude ran this exact command or file in a Linux sandbox (Ubuntu 24.04 container and Linux host, 4 cores, 16 GB) and saw it work. Output shown is real.
[Not tested on Windows]	Windows-only step written from official Microsoft/OGS documentation. Claude could not run Windows. You test it.
[Verified by the user on Windows 10 Pro (build 19045) with WSL2]	The user ran this step on the lab PC (Windows 10 Pro build 19045, WSL2) and it worked.
TO-VERIFY	A value or statement Claude could not confirm. Do not cite it until you have checked it.
Tip / Warning / Check / Why this matters	Call-out boxes. “Why this matters” gives the physics or numerical reason for a step.

Commands are in grey boxes. Type them exactly. Lines starting with # inside a box are comments. In every new Ubuntu terminal, first activate the Python environment:

```
cd ~
export PATH="$HOME/.local/bin:$PATH"
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
```

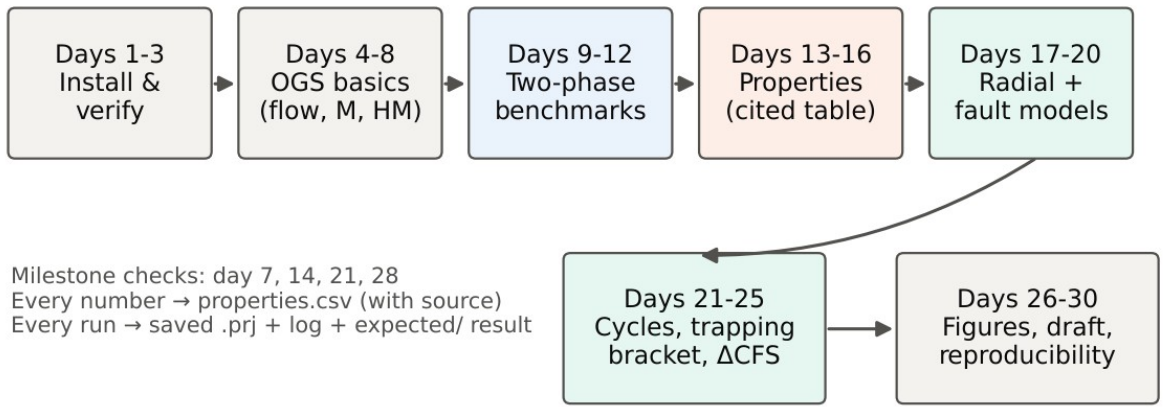


Figure 1. Workflow of the 30-day plan.

1. Feasibility and honesty

Before any modelling, five questions decide what this study can and cannot claim. Each answer below comes from the OGS source code (GitHub mirror ufz/ogs, master of 11 Nov 2026 and tag 6.5.9) and from test runs, not from memory.

1.1 Working title options

The original title promised dual-porosity and micro-pore trapping physics that OGS cannot resolve. Three honest alternatives:

1. **Hydro-mechanical screening of cyclic hydrogen storage in a depleted, microporous chert gas reservoir near critically stressed Midcontinent faults** (recommended; used on the title page).
2. **Safe operating windows for cyclic hydrogen storage in a depleted Mississippian chert reservoir: a coupled two-phase flow and geomechanics screening study**
3. **Pressure, trapping bounds and fault stress during seasonal hydrogen cycling in a high-porosity, low-permeability chert reservoir (Kansas analogue)**

1.2 Q1: Is dual porosity / dual permeability available for two-phase flow?

Answer: No. The two-phase processes (`TH2M`, `TWOPHASE_FLOW_PP`) contain no dual-continuum formulation. The only micro-porosity model in OGS belongs to `RICHARDS_MECHANICS` (a single mobile phase, written for swelling clays; property `saturation_micro`).

Best defensible approximation (used here): an effective single continuum. Micro-pore water is represented as immobile water (residual liquid saturation `res_S_Lr` = 0.45), the chert's permeability is an upscaled effective value, and the capillary curve represents the combined pore system. A second option for later is explicit zoning by MaterialIDs (fractured layers vs. matrix layers).

> **WARNING**
> Limits: matrix-fracture transfer rates, delayed drainage of micro-pores and any dual-porosity time-scale cannot be resolved. “Micro-pore trapping” can only appear through the residual saturations and the capillary curve, as a bounded estimate, never as a prediction.

1.3 Q2: Is there relative-permeability or capillary hysteresis?

Answer: No. No hysteresis model exists in the material library or in the two-phase processes (searched source code). All saturation and relative-permeability models are single-valued (van Genuchten, Brooks-Corey, Udell, Liakopoulos, generalized power, tabulated Curve).

Consequence: gas trapped behind a retreating gas front during withdrawal is not simulated, so a single run over-estimates recovery. Hysteresis is strong for H2-brine systems (Lysyy et al. 2022).

What we do: a trapping bracket. Bracket A uses drainage curves with residual gas saturation 0 (no trapping, upper bound on recovery). Bracket B uses curves whose residual gas saturation is the Land (1968) trapped saturation S_gr = S_gi/(1 + C S_gi) with S_gi = 1 - S_Lr, applied everywhere (maximum trapping, lower bound). With C = 2, S_gr = 0.262.

Tested size of this uncertainty: with Land coefficients C = 1, 2 and 5 the cycle-3 recovery is 0.12, 0.38 and 0.72, against 0.85 without trapping **[Tested by Claude on Linux]**. Trapping is therefore the largest single uncertainty, and the paper must say so.

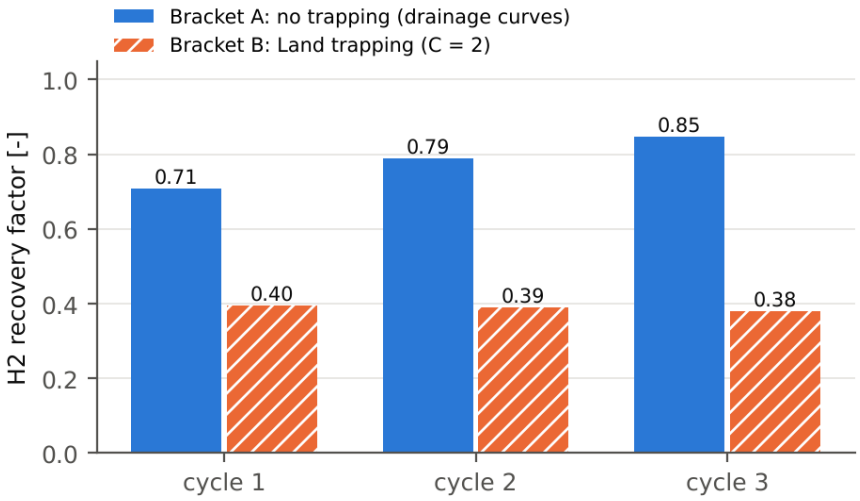


Figure 2. Recovery factor per cycle for the two trapping brackets (starter radial model, synthetic properties). **[Tested by Claude on Linux]**

1.4 Q3: Are hydrogen properties built in?

Answer: No. There is no hydrogen entry in the OGS material library; you supply every H2 property. What the starter does:

Property	In the model	Source / evidence
Gas density	`IdealGasLawBinaryMixture` (ideal gas of H2 + vapour)	Peng-Robinson (`PengRobinson`) and `IdealGasLaw` abort in TH2M with phase transition: "dValue is implemented for derivatives with respect to gas phase pressure or temperature only" **[Tested by Claude on Linux]**
Density error	+2.2 % at 4 MPa, +4.5 % at 8 MPa, +5.7 % at 10 MPa	vs. reference EOS of Leachman et al. (2009) via CoolProp; starter/properties/h2_properties_check.txt **[Tested by Claude on Linux]**
Viscosity	9.5e-6 Pa s (constant)	Muzny et al. (2013) via CoolProp; varies < 2 % over 4-15 MPa **[Tested by Claude on Linux]**
Solubility (Henry)	7.8e-6 mol/(m3 Pa)	Sander (2023), pure water at 298 K. Brine salting-out and temperature ignored **TO-VERIFY**
Dissolved H2 mass	brine density depends on `concentration`	Without it dissolved mass was zero (tested); same as OGS benchmark bourgeat.prj **[Tested by Claude on Linux]**

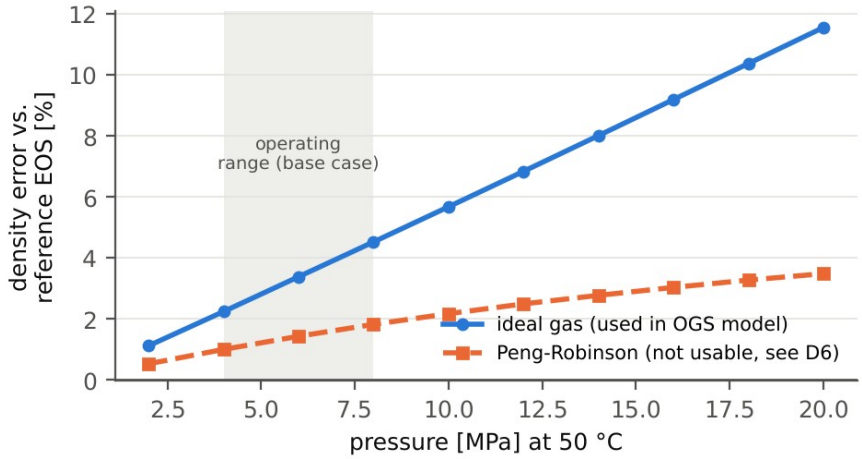


Figure 3. Error of the H2 density models usable in OGS versus the reference equation of state, 50 °C. **[Tested by Claude on Linux]**

1.5 Q4: How can a fault be represented, and how is Coulomb stress computed?

Interface elements are not available for two-phase flow. Lower-dimensional interface elements (LIE) exist only for `SMALL_DEFORMATION_WITH_LIE` and `HYDRO_MECHANICS_WITH_LIE` (single phase). For TH2M the fault is a **thin band of ordinary elements** (10 m wide, MaterialID 4) with its own permeability and stiffness. **Slip is not modelled.**

Coulomb stress change is computed afterwards in Python (starter/04_fault_section/coulomb.py) on each fault-band cell:

```
CFF   = tau - mu * sigma_n'                (compression positive, cohesion 0)
dCFS  = d(tau along background slip direction) - mu * d(sigma_n')
sigma_n' = sigma_n - p_eff,   p_eff = p_g - S_L * p_c   (Bishop, chi = S_L, biot = 1)
```

The background stress comes from an Andersonian stress state (Sv, SHmax, Shmin, SHmax azimuth) rotated into the section; the change comes from OGS. OGS `sigma` is the tension-positive Bishop effective stress; the run confirmed sigma - sigma_total = p_g - S_L p_c to within 0.3 kPa **[Tested by Claude on Linux]**. Six verification tests with hand-calculated answers pass (Mohr circle, pore-pressure-only change, Andersonian optimal dip 60.48° for mu = 0.6, rotation, OGS tensor order) **[Tested by Claude on Linux]**.

1.6 Q5: What is already published, and what is new?

Work	What it did	Relevance
Pfeiffer & Bauer (2015); Pfeiffer (2017, PhD)	H2 storage in porous media, OGS coupled with ECLIPSE (multi-component gas)	Closest OGS-related UHS work; flow-focused, sandstone
Grunwald et al. (2022); Pitz et al. (2023)	TH2M formulation in OGS and its benchmarks (nuclear waste)	Verification basis; not storage
Hogeweg et al. (2022)	UHS benchmark (ECLIPSE, DuMux)	Generic sandstone reservoir
Ramesh Kumar et al. (2023)	Review of geomechanics of UHS (depleted fields, caverns)	Fault reactivation context
Lysyy et al. (2022)	Measured H2 relative-permeability hysteresis	Justifies the trapping bracket

Two web searches did not find a published TH2M (OpenGeoSys) study of hydrogen storage in a depleted reservoir with fault stress. A search is not proof of absence; check Scopus and Google Scholar again before submission **TO-VERIFY**.

> **WARNING**
> "First study with OGS" is a weak novelty claim (it is about software, not science). The sharpened claim approved for this study: a hydro-mechanical safe operating window for cyclic H2 storage in high-porosity, low-permeability, high-water-saturation chert near critically stressed Midcontinent faults, with log-derived heterogeneity and bracketed (not predicted) trapping. The strength of the paper will come from the chert setting, the bracketed uncertainty, and the coupling of recovery with fault stress.

1.7 The three limits (repeat these in the manuscript)

#	Limit	Exact wording to use
1	No hysteresis	Trapping is reported only as a bracket: one run with the drainage curve (no trapping) and one with an imbibition curve carrying a Land-type trapped-gas saturation. Neither is a prediction.
2	No gas mixing	"Cushion gas fraction" means hydrogen left in place as cushion. Purity and mixing with methane or nitrogen are not modelled. Residual native gas is represented by the same gas component (H2).
3	Effective-continuum chert	Micro-pore effects enter only through effective properties (immobile water, upscaled permeability, capillary curve). Matrix-fracture transfer is not resolved.

Further simplifications (also stated in the manuscript): isothermal (temperature fixed); water evaporation off (vapour_pressure = 0); ideal-gas H2 density (+2 to +6 % in the operating range); 2D geometry (axisymmetric well; plane-strain section); no gravity in the base case (tested: gravity changes cycle-1 recovery from 0.707 to 0.708 in the 20 m layer); linear elasticity; fault slip not modelled; no geochemistry or microbial H2 consumption; withdrawal at a bottom-hole gas pressure equal to the initial depleted pressure (gas-only producer).

1.8 Compute plan for your PC

> CHECK
>
> **My machine (lab PC):** Windows 10 Pro build 19045, AMD Ryzen 7 5700X (8 cores / 16 threads), 32 GB RAM, virtualization enabled, WSL2 + Ubuntu, Python 3.12 venv with OGS 6.5.9 **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**.
>
> The models in this guide are sized to run on this hardware: 2D axisymmetric (well) and 2D plane-strain (fault) models first; 3D only as an optional later step.

Run times measured in the 4-core Linux sandbox (the 8-core lab PC should be similar or faster):

Model	Cells	Run time	Memory
Six official benchmarks	small	< 1.5 min total	< 1 GB
Radial model, 1 cycle	600	≈ 2.5 min	< 1 GB
Radial model, 3 cycles (drainage / imbibition)	600	≈ 5 / 8 min	< 1 GB
Fault section, 1 cycle	9 372	≈ 19 min	≈ 1-2 GB
Coarse 3D sector (optional, not built)	50k-200k	hours (estimate)	8-24 GB (estimate) **TO-VERIFY**

Plan: all paper runs in 2D. Run two OGS jobs at the same time (one per brackets/scenario), each using 4 threads (`export OMP_NUM_THREADS=4`; 8 physical cores on the lab PC). Each run writes 50-150 MB, so keep about 20 GB free and delete `out*/` folders you no longer need. 3D is optional and only after the 2D results are complete.

1.9 What was verified, and how

Item	Status
pip install ogstools[ogs]==0.8.2 → ogs 6.5.9 (Linux, Python 3.13 host and 3.12 container)	**[Tested by Claude on Linux]**
System libraries needed on clean Ubuntu 24.04 (libgl1, libglu1-mesa, libgomp1, X libs, libosmesa6)	**[Tested by Claude on Linux]** (found by container test)
check_install.py, six official benchmarks (all PASS)	**[Tested by Claude on Linux]**
Radial TH2M model: 3 cycles x 2 brackets; mass balance < 1 %; 1-cycle run identical in container (RF 0.707)	**[Tested by Claude on Linux]**
Fault section TH2M + Coulomb analysis + 6 verification tests	**[Tested by Claude on Linux]**
Docker container (starter/docker/Dockerfile) build and check	**[Tested by Claude on Linux]** on Linux; Docker Desktop on Windows **[Not tested on Windows]**
WSL2 + Ubuntu install, apt packages, uv + Python 3.12 venv, ogstools[ogs] (ogs 6.5.9, ogstools 0.8.2), repository clone	**[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
System libraries + check_install.py on Windows, .wslconfig, VS Code WSL extension, ParaView on Windows, native Windows pip	**[Not tested on Windows]**
Site values (depth, temperature, brine, stress magnitudes, fault geometry)	**TO-VERIFY** synthetic/literature

2. Installing OGS on Windows 10 or 11

OGS (OpenGeoSys) is an open-source finite-element program for coupled thermal (T), hydraulic (H) and mechanical (M) processes in porous media. **ogstools** is its Python companion (meshes, running, reading results). The pip package `ogstools[ogs]` installs both, plus gmsh (mesher) and pyvista (3D data).

Route	When to use	MFront / PETSc	Status
A. WSL2 + Ubuntu + uv + Python 3.12 venv (primary)	Default for this study	MFront yes / PETSc no	**[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
A2. WSL2 + system Python venv	Only if `python3 --version` is 3.10-3.13 (e.g. Ubuntu 24.04)	MFront yes / PETSc no	**[Tested by Claude on Linux]** on Linux; Windows **[Not tested on Windows]**
B. Native Windows pip	If WSL is not allowed on your PC	no / no (official table)	**[Not tested on Windows]**
C. Docker container	Reproducibility package; fallback	MFront yes / PETSc no	**[Tested by Claude on Linux]** on Linux; Docker Desktop **[Not tested on Windows]**

MFront is a library for advanced rock constitutive models (not needed for the linear-elastic starter). PETSc enables parallel MPI runs (not needed on a laptop).

2.1 Check your PC

```
# PowerShell (Windows)
Get-CimInstance Win32_ComputerSystem | Select TotalPhysicalMemory
Get-CimInstance Win32_Processor | Select NumberOfCores,NumberOfLogicalProcessors
Get-PSDrive C
winver
```

[Not tested on Windows] `wsl --install` needs Windows 10 build 19041 or later, or Windows 11. Minimum for this plan: 16 GB RAM, 4 cores, 30 GB free. The lab PC (build 19045, 8 cores, 32 GB) is comfortable for all 2D work.

2.2 Route A: WSL2 + Ubuntu + uv + Python 3.12 (primary, verified on Windows)

WSL2 (Windows Subsystem for Linux) runs a real Linux kernel inside Windows; the OGS team recommends it for Windows users. **uv** is a fast Python installer that can create a virtual environment with a specific Python version. This exact sequence was run on the lab PC (Windows 10 Pro build 19045).

1. PowerShell as Administrator: install WSL and Ubuntu (download takes a few minutes); create the Linux user and password. A question about sending usage metrics to Canonical can be answered y or n **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

   ```
   wsl --install -d Ubuntu
   ```

2. In Ubuntu, start in the Linux home folder (new windows can open in /mnt/c/Windows/system32) and install basic tools. A red "Job for systemd-binfmt.service failed" message is harmless in WSL **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
cd ~
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git curl
```

3. **Check the Python version before installing OGS** **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
python3 --version
```

   The OGS pip package supports Python 3.10-3.13 only. The current Ubuntu release ("resolute") ships Python 3.14 (3.14.4 on the lab PC), where `ogs` cannot be installed. Do not use the system Python then: create a Python 3.12 environment with uv.

4. Install uv, create and activate the Python 3.12 environment, install OGS **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv venv --python 3.12 ~/ogs-env
source ~/ogs-env/bin/activate
python --version
uv pip install "ogstools[ogs]"
```

5. Verify. Expected: `ogs  version: 6.5.9` (build options include `OGS_USE_MFRONT=ON`); ogstools 0.8.2 located in ~/ogs-env; `ogs --help` prints its usage text **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
ogs --version
uv pip show ogstools
ogs --help
```

6. Clone the starter repository into ~/work (create folders before `cd`) **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
mkdir -p ~/work/ogs-tests
cd ~/work
git clone https://github.com/abrarmahir31/h2-chert-storage-ogs.git
```

7. System libraries (gmsh, off-screen plots), CoolProp (Day 13) and the install check. Tested by Claude on Linux **[Tested by Claude on Linux]**; not yet run on Windows **[Not tested on Windows]**:

```
sudo apt install -y libgl1 libglu1-mesa libgomp1 libxcursor1 libxinerama1 \
    libxft2 libxrender1 libosmesa6
uv pip install CoolProp==8.0.0
cd ~/work/h2-chert-storage-ogs
python starter/00_check_install/check_install.py
```

8. Optional: limit WSL resources with `C:\Users\<you>\.wslconfig` (e.g. `memory=24GB`, `processors=8` under `[wsl2]`), then `wsl --shutdown` [Not tested on Windows].

Start every new Ubuntu session with [Verified by the user on Windows 10 Pro (build 19045) with WSL2]:

```
cd ~
export PATH="$HOME/.local/bin:$PATH"
source ~/ogs-env/bin/activate
```

Route A2 (alternative): system Python, only if it is 3.10-3.13

On Ubuntu 24.04 (Python 3.12) the plain venv also works. Tested by Claude on Linux [Tested by Claude on Linux]; not tested on Windows [Not tested on Windows]:

```
python3 -m venv ~/ogs-env
source ~/ogs-env/bin/activate
pip install --upgrade pip
pip install -r ~/work/h2-chert-storage-ogs/starter/requirements.txt
```

> **TIP**
> Keep your work in the Linux file system (~/...), not under /mnt/c/. File access across the Windows boundary is much slower, and OGS writes many files.

2.3 VS Code and ParaView

- VS Code: install on Windows, add the **WSL** extension, then in Ubuntu run `code .` inside the repository [Not tested on Windows].
- ParaView (paraview.org, Windows installer): open results through `\\wsl.localhost\Ubuntu\home\<user>\work\...`. Running `explorer.exe .` in Ubuntu opens the current folder in Windows Explorer [Not tested on Windows].

2.4 Route B: native Windows pip (not tested)

From the official OGS install page (Python 3.10-3.13 required). Not tested on Windows [Not tested on Windows]:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1      # PowerShell
pip install ogstools[ogs]
```

If PowerShell refuses to run Activate.ps1, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once [Not tested on Windows]. The bash scripts of the starter (fetch_benchmarks.sh) need Git Bash on this route.

2.5 Route C: Docker container (not tested on Windows)

starter/docker/Dockerfile builds Ubuntu 24.04 + the system libraries + the pinned pip packages. On Windows use Docker Desktop with the WSL2 backend [Not tested on Windows]. Commands [Tested by Claude on Linux]:

```
cd ~/work/h2-chert-storage-ogs
docker build -t ogs-h2 -f starter/docker/Dockerfile .
docker run --rm -v "$PWD:/work" -w /work ogs-h2 \
    python starter/00_check_install/check_install.py
```

The OGS project also provides Apptainer (Singularity) images for Linux (`apptainer exec ogs-6.x.x-serial.squashfs ogs ...`, see the container page of the OGS user guide) [Not tested on Windows].

2.6 Verify the installation with real runs

Quick version checks (verified on the lab PC) [Verified by the user on Windows 10 Pro (build 19045) with WSL2]: `python --version` (3.12.x inside the venv), `ogs --version` (6.5.9), `uv pip show ogstools` (0.8.2). Then the full check:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
python starter/00_check_install/check_install.py
```

Real output from the sandbox (your CPU/RAM lines will differ) [Tested by Claude on Linux]:

```
Python                      3.13.18                                 OK
Logical CPU cores           4                                       OK
RAM visible here [GB]       15.7                                    OK
import ogstools             0.8.2                                   OK
import gmsh                 4.15.2                                  OK
ogs executable on PATH      /home/user/work/.venv/bin/ogs           OK
ogs --version               6.5.9                                   OK
test run (ogstools example) 11 time steps written                   OK
pressure range [MPa]        29.00 .. 31.00                          OK

RESULT: ALL OK
```

Then run the six official benchmarks (Day 3): `bash fetch_benchmarks.sh` and `python run_benchmarks.py` in starter/02_benchmarks. All six must PASS.

2.7 Updating OGS

With the uv route (not yet tested on Windows [Not tested on Windows]):

```
source ~/ogs-env/bin/activate
uv pip install --upgrade "ogstools[ogs]"
ogs --version
python starter/00_check_install/check_install.py
cd starter/02_benchmarks
bash fetch_benchmarks.sh <new-version-tag>
python run_benchmarks.py
```

WARNING

Never update OGS in the middle of the paper runs. Finish a batch, then update, rerun the benchmarks and the 1-cycle radial test, and compare with starter/*/expected/. Record the new versions in starter/pip_freeze_tested.txt and DECISIONS.md.

2.8 Most common errors and how to fix them

Error message / symptom	Cause	Fix
`ogs: command not found`	venv not active	`source ~/ogs-env/bin/activate` **[Tested by Claude on Linux]**
pip/uv cannot find `ogs`; `python3 --version` shows 3.14	Python 3.14 is not supported (3.10-3.13)	`uv venv --python 3.12 ~/ogs-env` (2.2) **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
`AttributeError` from `print(ogstools._version_)`	one underscore typed; it is `__version__` (two on each side)	use `uv pip show ogstools` instead **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
`cd ~/work`: No such file or directory	`cd` before the folder exists	`mkdir -p ~/work` first, then `cd ~/work` **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
Prompt shows /mnt/c/Windows/system32	new Ubuntu windows start in the Windows folder	`cd ~` before anything else **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
Red "Job for systemd-binfmt.service failed" during apt	WSL has no full systemd boot here	harmless: ignore it **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**
`uv: command not found`	uv's folder not on PATH in this terminal	`export PATH="$HOME/.local/bin:$PATH"`
`OSError: libGLU.so.1` (or `libGL.so.1`, `libgomp.so.1`) on `import gmsh`	missing system libraries	apt install line of 2.2 **[Tested by Claude on Linux]**
`libOSMesa not found` / crash when plotting	no off-screen renderer	`sudo apt install libosmesa6`; `export PYVISTA_OFF_SCREEN=true` **[Tested by Claude on Linux]**
pip: no matching distribution for ogs	Python outside 3.10-3.13	create a Python 3.12 venv with uv (2.2) **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**

Error message / symptom	Cause	Fix
`Mismatch in numerical Jacobian perturbation configuration`	`<epsilons>` count wrong	TH2M: 3 values (p_g, p_c, T) **[Tested by Claude on Linux]**
`Key <x> has been read 0 times` / `read 1 time(s) less`	misspelled/unused XML tag (or follow-up of an earlier error)	fix the first error in the log; copy tags from working files
`The required bulk node ids map does not exist in the boundary mesh`	a mesh used for a BC lacks `bulk_node_ids`	create boundary meshes with ogstools; add ids to the domain if it is used as a BC mesh **[Tested by Claude on Linux]**
`PengRobinson::dValue is implemented for derivatives ...`	PR not supported with TH2M phase transition	use `IdealGasLawBinaryMixture` (D6) **[Tested by Claude on Linux]**
`Newton: The linear solver failed in the compute() step` at step 1	gas phase absent in parts of the domain (negative p_c) / singular system	uniform initial p_c, high p_b in seals (D12) **[Tested by Claude on Linux]**
Many rejected time steps	too large steps / sharp schedule switches	reduce maximum_dt, initial_dt; keep switches as output times
PowerShell: running scripts is disabled	execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` **[Not tested on Windows]**
Very slow file access	working under /mnt/c	work in ~/ inside WSL

3. Setting up OGS for this study

3.1 Project folder layout

```
h2-chert-storage-ogs/
├── README.md, PROGRESS.md, DECISIONS.md
├── docs/  OGS_H2_Storage_Aetna_30Day_Guide.docx (+ .pdf)
└── starter/
    ├── requirements.txt          pinned versions
    ├── properties/               properties.csv  ← ALL numbers + sources
    ├── common/                   props.py, schedule.py
    ├── 00_check_install/         check_install.py
    ├── 02_benchmarks/            fetch + run 6 official benchmarks
    ├── 03_radial_h2/             axisymmetric well model (TH2M)
    │   ├── make_mesh.py → build_prj.py → ogs → postprocess.py
    │   └── expected/             results to compare with
    └── 04_fault_section/         plane-strain section + fault band
        ├── make_mesh.py → build_prj.py → ogs → coulomb.py
        └── test_coulomb.py       verification (known answers)
```

Figure 4. Repository layout. Only properties.csv holds numbers; scripts read it.

3.2 How a project file (.prj) is organised

A `.prj` file is an XML text file that tells OGS everything about one simulation. Below are the real blocks of starter/03_radial_h2/radial_drainage.prj (shortened).

<meshes>

Which mesh files to read. The first is the domain; the others are boundary meshes used by boundary conditions. `axially_symmetric="true"` makes the 2D (r, z) mesh a revolved body.

```
<meshes>
    <mesh axially_symmetric="true">mesh/domain.vtu</mesh>
    <mesh axially_symmetric="true">mesh/well.vtu</mesh>
</meshes>
```

<processes>

Which physics (process type) and its settings: the constitutive model of the solid, the process variables, extra outputs (secondary variables), gravity (`specific_body_force`).

```
<process>
    <name>TH2M</name><type>TH2M</type><integration_order>2</integration_order>
    <constitutive_relation><type>LinearElasticIsotropic</type>
        <youngs_modulus>E</youngs_modulus><poissons_ratio>nu</poissons_ratio>
    </constitutive_relation>
    <jacobian_assembler><type>CentralDifferences</type>
        <epsilons>0.01 0.01 1e-06</epsilons></jacobian_assembler>
    <specific_body_force>0 0</specific_body_force>
</process>
```

<media>

Material properties: one medium per MaterialID, each with phases (AqueousLiquid, Gas, Solid), components (H2, W) and medium-level properties (permeability, porosity, saturation curve, relative permeabilities, Biot coefficient, Bishop stress).

```
<property><name>saturation</name><type>SaturationVanGenuchten</type>
    <residual_liquid_saturation>0.45</residual_liquid_saturation>
    <residual_gas_saturation>0.0</residual_gas_saturation>
    <p_b>500000.0</p_b><exponent>0.5</exponent></property>
```

<time_loop>

How to step through time (time stepping scheme, convergence criterion, non-linear solver) and what to write out (output).

```
<time_stepping><type>IterationNumberBasedTimeStepping</type>
    <t_initial>0</t_initial><t_end>95904000.0</t_end>
    <initial_dt>60</initial_dt><minimum_dt>1</minimum_dt>
    <maximum_dt>172800</maximum_dt>
    <number_iterations>2 4 8 12 20</number_iterations>
    <multiplier>1.6 1.3 1.0 0.7 0.5</multiplier></time_stepping>
```

<parameters / curves>

Named values used elsewhere (Young's modulus, initial values) and time curves (the injection schedule).

```
<parameter><name>h2_flux</name><type>CurveScaled</type>
    <curve>injection_schedule</curve><parameter>one</parameter></parameter>
```

<process_variables>

The unknowns (gas_pressure, capillary_pressure, temperature, displacement) with initial conditions and boundary conditions.

```
<boundary_condition><mesh>well</mesh><type>Neumann</type>
    <component>0</component><parameter>h2_flux</parameter></boundary_condition>
<boundary_condition><mesh>well</mesh><type>DirichletWithinTimeInterval</type>
    <parameter>p_bhp_min</parameter>
    <time_interval><start>28512000.0</start><end>38880000.0</end></time_interval>
</boundary_condition>
```

<nonlinear_solvers / linear_solvers>

Newton settings and the linear solver (here the direct sparse solver SparseLU of the Eigen library).

```
<nonlinear_solver><name>basic_newton</name><type>Newton</type>
    <max_iter>25</max_iter><linear_solver>direct</linear_solver></nonlinear_solver>
```

> **WHY THIS MATTERS**
> OGS stops if a tag is misspelled or unused. Never type tags from memory: copy them from a working file (benchmarks or the starter). All tags in this guide come from files that ran with OGS 6.5.9.

3.3 Making meshes

A **mesh** divides the domain into small cells (elements); OGS solves the equations on them. The starter uses the **gmsh** Python API: define points, lines and surfaces, mark **physical groups** (named sets such as `well`), mesh, then `ogstools.Meshes.from_gmsh(..., dim=2)` writes the domain and one boundary mesh per group, with the `bulk_node_ids` and `bulk_element_ids` arrays OGS needs **[Tested by Claude on Linux]**.

```
import ogstools as ot
meshes = ot.Meshes.from_gmsh("mesh/radial.msh", dim=2, log=False)
meshes.save("mesh", overwrite=True)      # domain.vtu, well.vtu, outer.vtu, ...
```

3.4 Running a simulation

```
ogs radial_drainage.prj -o out > out/radial_drainage.log 2>&1
ogs --help        # -o output dir, -m mesh dir, -l log level, -p xml patch, --version
```

Output: one `.vtu` file per output time plus a `.pvd` index file that ParaView and ogstools read as a time series.

3.5 Post-processing

ParaView

1. File > Open the `.pvd`, press Apply.
2. Choose a variable (saturation, gas_pressure, sigma) in the colour menu; press play to animate.
3. Filters > Plot Over Line for profiles; Filters > Warp By Vector (displacement) to see deformation; Filters > Threshold on MaterialIDs to isolate the fault band.

ogstools in Python {T}

```
import ogstools as ot
ms = ot.MeshSeries("out/radial_drainage.pvd")
print(ms.timevalues[-1] / 86400, "days")
m = ms[-1]                                    # last output (a pyvista mesh)
print(m.point_data["gas_pressure"].max() / 1e6, "MPa")
```

3.6 Choosing the process

Process type (verified name)	Physics	Use in this study
`LIQUID_FLOW`	single-phase flow	Day 4 learning (Theis)
`HYDRO_MECHANICS`	single-phase flow + mechanics	Day 6 learning; single-phase approximation only
`HYDRO_MECHANICS_WITH_LIE`	single-phase HM with interface elements	Only for a single-phase fault-slip side study
`TWOPHASE_FLOW_PP`	two-phase flow, no mechanics	Optional cross-check of flow
`RICHARDS_MECHANICS`	unsaturated single mobile phase + mechanics (has micro-porosity)	Not suitable: gas is not a mobile phase
`TH2M`	two-phase two-component flow + heat + mechanics, dissolution	Main process (run isothermally)

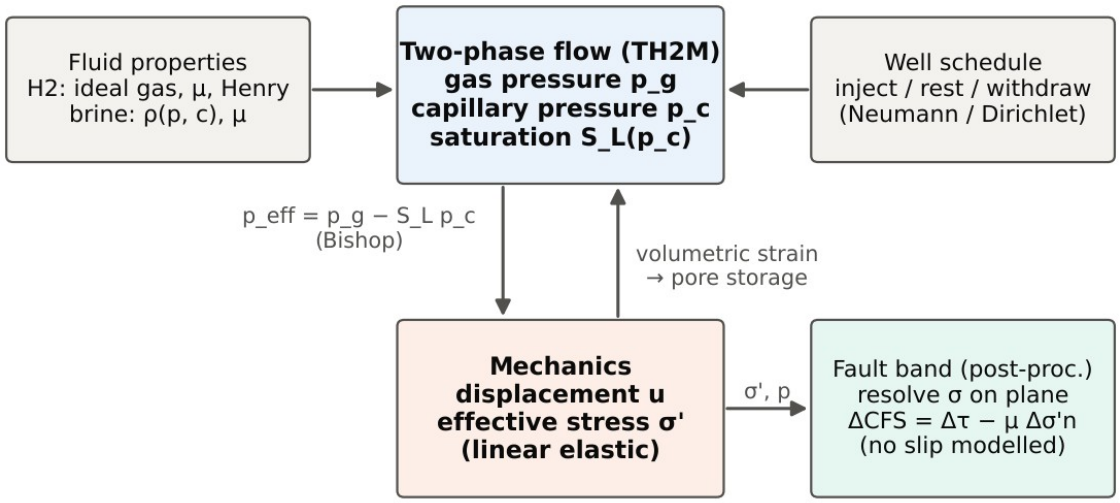


Figure 5. Couplings in the model: flow and mechanics in TH2M, fault stress in post-processing.

3.7 The starter models

3.7.1 Radial well model (starter/03_radial_h2)

- Geometry: 2D axisymmetric, r = 0.1-500 m (closed compartment), 20 m thick, 600 quadrilaterals refined toward the well.
- Initial state: depleted, p_g = 4 MPa, S_L = 0.60 (p_c = 1.764 MPa from van Genuchten), 50 °C.
- Well: H2 injection 0.1 kg/s as a Neumann mass flux; withdrawal by `DirichletWithinTimeInterval` gas pressure = 4 MPa (gas-only producer, never below the initial pressure, so native gas is not produced).
- Schedule (starter/common/schedule.py): 180 d cushion filling, 30 d rest, then 3 x (120 d injection, 30 d rest, 120 d withdrawal, 30 d rest).
- Recovery factor (per cycle) = H2 produced in the withdrawal window / H2 injected in that cycle. Cushion fraction = cushion mass / (cushion + working mass) = 0.60. This per-cycle RF can exceed 1

when withdrawal also produces cushion gas (seen with a 360-day cushion: RF 1.19), so always report it together with the H2 in place after each cycle **[Tested by Claude on Linux]**.

- Self-check: H2 inventory change vs. injected mass before the first withdrawal; error < 1 % **[Tested by Claude on Linux]**.

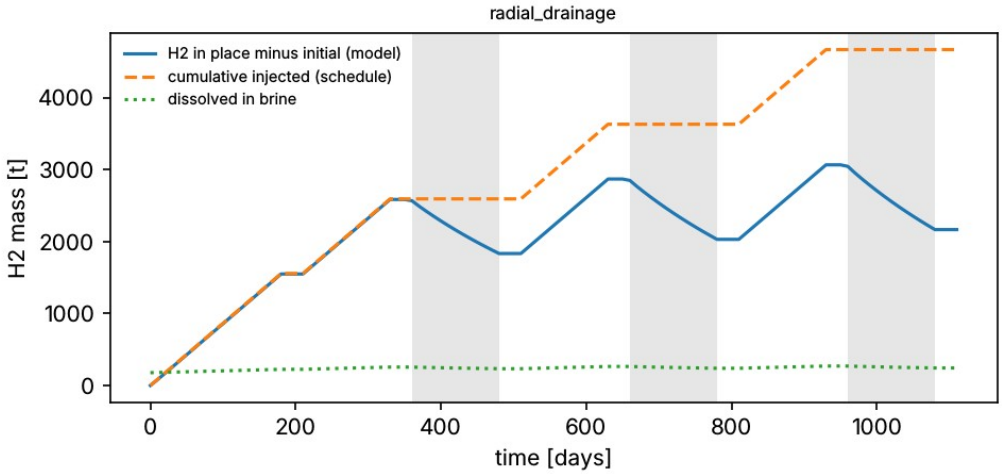


Figure 6. H2 inventory vs. injected mass, drainage bracket (grey = withdrawal). **[Tested by Claude on Linux]**

3.7.2 Fault section (starter/04_fault_section)

- Geometry: 2D plane strain, 3 km long, surface to 2.5 km depth; overburden, 50 m caprock, 20 m reservoir at 1450 m, underburden; 10 m fault band dipping 60° crossing the reservoir 600 m from the well (left boundary = symmetry plane).
- Initial capillary pressure uniform (capillary equilibrium); seals stay > 99.5 % brine-saturated because of their high p_b (D12).
- Stress: changes from the depleted initial state (no gravity, no initial stress); in-situ stress added in coulomb.py (superposition).
- Output: dCFS history and profile, plus a sensitivity table for friction, dip, Shmin/Sv, SHmax/Sv and SHmax azimuth. The dip entries only rotate the plane at the same cells; a full re-mesh at 45 degrees gave 0.92 MPa instead of 1.23 MPa, so re-mesh for reported dip results **[Tested by Claude on Linux]**.

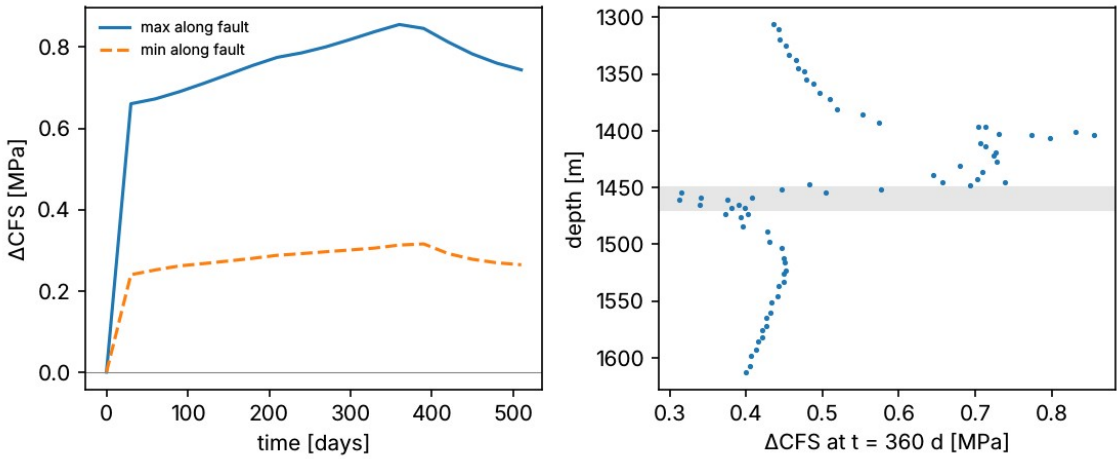


Figure 7. Coulomb stress change on the fault band (synthetic stress state). **[Tested by Claude on Linux]**

CHECK

Base case: max dCFS = +0.86 MPa at the end of the first injection; the most critical fault cell goes from CFF0 = -0.29 MPa to +0.55 MPa. With Shmin/Sv = 0.6 or mu = 0.4 the synthetic fault already exceeds failure before injection (CFF0 > 0), which is physically inconsistent and shows that the stress state must be calibrated before any conclusion about fault safety.

4. The 30-day plan

30 consecutive days, 6 hours per day, 180 hours. Changes from the suggested arc, and why: (1) the fault model is a 2D plane-strain section rather than a 3D sector, because fault stress depends more on vertical layering than on the third dimension, and 2D fits your laptop; (2) days 13-16 are a dedicated property block, because OGS has no hydrogen data and every number must be cited; (3) model building (days 17-20) therefore starts one day later than suggested.

Days	Block	Main result
1-3	Install and verify; official benchmarks	ALL OK + 6 x PASS
4-8	OGS basics: flow, mechanics, HM, meshing, time stepping	3 small models understood; radial mesh built (milestone day 7)
9-12	Two-phase and TH2M verification (McWhorter, Liakopoulos, Bourgeat)	3 verification notes
13-16	Property block (fluids, chat, curves, mechanics/stress)	Sourced properties.csv (milestone day 14)
17-20	Radial well model and fault section	Both models run; Coulomb analysis
21-25	Cushion, trapping, sensitivity, safe window, convergence	Results database (milestone day 21)
26-30	Figures, tables, methods, manuscript, reproducibility, review	Manuscript skeleton (milestone day 28)

Day 1 – Check your PC and switch on WSL2 — Week 1 · 6 h

Goal. Know your hardware limits and have a working Ubuntu terminal (WSL2) on Windows 10 (build 19041 or later) or Windows 11.

Block	Hours	What you do
Reading	1.0	Guide sections 1.8 and 2.1-2.2; Microsoft WSL install page
Hands-on	3.0	Hardware and build check, WSL2 + Ubuntu install, optional .wslconfig
Testing	1.0	Restart, open Ubuntu, check cores/memory seen by Linux
Notes	1.0	Start your lab notebook (notes/day01.md)
Total	6.0	

> **WHY THIS MATTERS**
> OGS runs natively on Linux. WSL2 is a real Linux kernel inside Windows, so the Linux pip package (with MFront support, which the native Windows pip package lacks) runs at near-native speed. Knowing your RAM and cores tells you how big a mesh you can afford.

Steps

1. Open **PowerShell** and check RAM, cores and the Windows build. Windows 10 needs build 19041 or later for `wsl --install` (the lab PC has 19045) **[Not tested on Windows]**:

```
Get-CimInstance Win32_ComputerSystem | Select TotalPhysicalMemory
Get-CimInstance Win32_Processor | Select NumberOfCores,NumberOfLogicalProcessors
Get-PSDrive C
winver
```

2. Check that virtualization is on: Task Manager > Performance > CPU > “Virtualization: Enabled” **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**.
3. Open PowerShell **as Administrator** and install WSL with Ubuntu. The download takes a few minutes; restart Windows if asked **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
wsl --install -d Ubuntu
```

4. Ubuntu opens and asks for a new Linux user name and password (write them down). If it asks whether to send usage metrics to Canonical, answering y or n are both fine **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**.
5. Optional: limit WSL resources. Create `C:\Users\<you>\.wslconfig` with Notepad (example for 32 GB / 8 cores), then run `wsl --shutdown` in PowerShell **[Not tested on Windows]**:

```
[wsl2]
memory=24GB
processors=8
```

6. In PowerShell check the WSL version [Not tested on Windows]: `wsl -l -v` must show VERSION 2.
7. In Ubuntu, go to your Linux home first (new windows can start in /mnt/c/Windows/system32), then check what Linux sees [Tested by Claude on Linux]:

```
cd ~
nproc
free -g
df -h ~
```

What correct output looks like

- `wsl -l -v` lists Ubuntu with VERSION 2.
- `nproc` prints 16 on the Ryzen 7 5700X (8 with processors=8); `free -g` shows about half your RAM by default (about 23-24 GB with memory=24GB).

Common mistakes and fixes

Mistake / symptom	Fix
`wsl --install` says virtualization is disabled	Enable it in the BIOS/UEFI (AMD: "SVM Mode"; Intel: VT-x), then retry [Not tested on Windows].
Ubuntu shows VERSION 1	`wsl --set-version Ubuntu 2` in PowerShell [Not tested on Windows].
Ubuntu prompt shows /mnt/c/Windows/system32	Type `cd ~` first; work only inside the Linux home folder [Verified by the user on Windows 10 Pro (build 19045) with WSL2].
`free -g` ignores .wslconfig	The file must be exactly `.wslconfig` (no .txt) in your Windows user folder, followed by `wsl --shutdown`.

Done when

- ☐ Hardware numbers and Windows build written in notes/day01.md
- ☐ Ubuntu runs as WSL version 2
- ☐ Linux sees your cores and memory

Save: notes/day01.md with hardware numbers and the output of `nproc`, `free -g`, `df -h ~`.

> **TIP**
>
> **If you only have 3 hours:** Run `wsl --install -d Ubuntu`, restart, create the Linux user and confirm Ubuntu opens.

Day 2 – Install Python 3.12 (uv), OGS, VS Code and ParaView — Week 1 · 6 h

Goal. Have OGS 6.5.9 + ogstools 0.8.2 in a Python 3.12 virtual environment inside WSL, clone the starter repository, and run the install check.

Block	Hours	What you do
Reading	0.5	Guide 2.2-2.6
Hands-on	3.0	apt packages, uv, Python 3.12 venv, ogstools[ogs], clone, VS Code, ParaView
Testing	1.5	Version checks, check_install.py; fix any FAIL
Notes	1.0	Record versions in your notes
Total	6.0	

> **WHY THIS MATTERS**
> A virtual environment (venv) is a private folder of Python packages, which keeps the exact OGS version of your study separate and reproducible. The current Ubuntu release ships Python 3.14, but the OGS pip package supports only Python 3.10-3.13, so pip cannot install it there. uv downloads a separate Python 3.12 just for this venv without touching the system Python.

Steps

1. Start in your Linux home folder [Verified by the user on Windows 10 Pro (build 19045) with WSL2]:

```
cd ~
```

2. Update Ubuntu and install basic tools [Verified by the user on Windows 10 Pro (build 19045) with WSL2]. A red message “Job for systemd-binfmt.service failed” can appear: it is harmless in WSL, ignore it.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git curl
```

3. **Check the Python version before installing anything** [Verified by the user on Windows 10 Pro (build 19045) with WSL2]:

```
python3 --version
```

   Python 3.10-3.13: the system Python would work. Python 3.14 (current Ubuntu “resolute” shows 3.14.4): do NOT use it for OGS. Either way, follow the uv route below; it was verified on Windows.

4. Install uv and make it available in this terminal [Verified by the user on Windows 10 Pro (build 19045) with WSL2]:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
```

5. Create and activate a Python 3.12 environment, then confirm its version **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
uv venv --python 3.12 ~/ogs-env
source ~/ogs-env/bin/activate
python --version
```

6. Install OGS and ogstools **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
uv pip install "ogstools[ogs]"
```

7. Verify **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
ogs --version
uv pip show ogstools
ogs --help
```

8. Clone the starter repository into a work folder in the Linux home (not under /mnt/c). Create the folder before you `cd` into it **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
mkdir -p ~/work/ogs-tests
cd ~/work
git clone https://github.com/abrarmahir31/h2-chert-storage-ogs.git
```

9. Still to do on Windows: system libraries for gmsh and off-screen plotting, CoolProp (needed on Day 13), then the install check. Tested by Claude on Linux **[Tested by Claude on Linux]**; not yet run on Windows **[Not tested on Windows]**:

```
sudo apt install -y libgl1 libglu1-mesa libgomp1 libxcursor1 libxinerama1 \
    libxft2 libxrender1 libosmesa6
uv pip install CoolProp==8.0.0
cd ~/work/h2-chert-storage-ogs
python starter/00_check_install/check_install.py
```

10. Install **VS Code** on Windows with the **WSL** extension; in Ubuntu run `code .` inside the repository **[Not tested on Windows]**.
11. Install **ParaView** (Windows installer from paraview.org); open results via `\\wsl.localhost\Ubuntu\home\<user>\work\...` in its file dialog **[Not tested on Windows]**.
12. Every new Ubuntu terminal starts with these three lines **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**:

```
cd ~
export PATH="$HOME/.local/bin:$PATH"
source ~/ogs-env/bin/activate
```

What correct output looks like

- `python --version` inside the venv: Python 3.12.x (3.12.16 on the lab PC).

- `ogs --version`: `ogs  version: 6.5.9`; the build options include `OGS_USE_MFRONT=ON`.
- `uv pip show ogstools`: Version 0.8.2, Location under ~/ogs-env.
- check_install.py (after the libraries): `RESULT: ALL OK` and pressure range `29.00 .. 31.00` MPa **[Tested by Claude on Linux]**.

Common mistakes and fixes

Mistake / symptom	Fix
pip/uv cannot find ogs; `python3 --version` shows 3.14	Python 3.14 is not supported (3.10-3.13 only). Use `uv venv --python 3.12` as above.
`AttributeError` from `python -c "import ogstools; print(ogstools._version_)"`	It needs two underscores on each side: `__version__`. Safer check: `uv pip show ogstools` **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**.
`cd ~/work`: No such file or directory	Create the folder first: `mkdir -p ~/work`, then `cd ~/work` **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**.
Files appear under /mnt/c/Windows/system32	New Ubuntu windows start there; run `cd ~` first **[Verified by the user on Windows 10 Pro (build 19045) with WSL2]**.
`uv: command not found`	Run `export PATH="$HOME/.local/bin:$PATH"` in that terminal.
`ogs: command not found`	The venv is not active: `source ~/ogs-env/bin/activate`.
`OSError: libGLU.so.1` / `libGL.so.1` / `libgomp.so.1`	Install the system libraries (step 9).

Done when

- ☐ `python --version` in the venv shows 3.12.x
- ☐ `ogs --version` shows 6.5.9
- ☐ `uv pip show ogstools` shows 0.8.2
- ☐ Repository cloned to ~/work/h2-chert-storage-ogs
- ☐ check_install.py: ALL OK
- ☐ VS Code and ParaView installed

Save: notes/day02.md with the outputs of `python --version`, `ogs --version`, `uv pip show ogstools` and check_install.py.

> **TIP**
>
> **If you only have 3 hours:** Do steps 1-8 (apt, Python check, uv, venv, ogstools, verify, clone). Libraries, check_install, VS Code and ParaView can wait until Day 3.

Day 3 – Run the official benchmarks

Week 1 · 6 h

Goal. Download six official OGS benchmarks matching your OGS version and run them all with their built-in checks against analytical or published results.

Block	Hours	What you do
Reading	1.0	Benchmark pages on opengeosys.org (Theis, McWhorter, Bourgeat)
Hands-on	2.5	fetch_benchmarks.sh, run_benchmarks.py, open results in ParaView
Testing	1.5	Confirm 6 x PASS; look at every saved figure
Notes	1.0	Table: benchmark, physics, what it checks, run time
Total	6.0	

> **WHY THIS MATTERS**
> A benchmark is a test problem with a known answer. If OGS reproduces it on your computer, your installation is correct, and you have a trusted starting file for every physics you will use later. The benchmarks are pinned to the git tag of your OGS version so the input files match the program.

Steps

1. Fetch the benchmark input files (about 30 MB) **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/02_benchmarks
bash fetch_benchmarks.sh
```

2. Run all six. `PYVISTA_OFF_SCREEN=true` lets the plotting library draw without a screen **[Tested by Claude on Linux]**:

```
export PYVISTA_OFF_SCREEN=true
python run_benchmarks.py
```

3. Open one result in ParaView: File > Open > `.../02_benchmarks/ogs_src/Tests/Data/TH2M/H2/mcWhorter/_out/result_McWhorter_H2.pvd`, click Apply, choose `saturation` in the colour menu, press the play button.
4. Compare your figures in `figures/` with the reference figures in `expected/`.

What correct output looks like

- Six lines ending in PASS: theis (~2 s), kirsch (~3 s), confined_compression (~5 s), mcwhorter (~17 s), liakopoulos (~8 s), bourgeat (~38 s) on a 4-core machine.
- figures/mcwhorter_0.png shows the numerical and exact saturation curves on top of each other.

Common mistakes and fixes

Mistake / symptom	Fix
`Benchmarks not found`	Run `bash fetch_benchmarks.sh` first, from inside starter/02_benchmarks.
kirsch crashes with an OSMesa message	Install `libosmesa6` (Day 2 step 1) and set `PYVISTA_OFF_SCREEN=true`.
git clone fails	Check internet access from WSL: `curl -I https://github.com`.

Done when

- ☐ 6 x PASS
- ☐ One .pvd opened and animated in ParaView
- ☐ Notes table filled

Save: figures/ folder (copy to notes/day03/) and the PASS list.

> **TIP**
>
> **If you only have 3 hours:** Run only `python run_benchmarks.py theis mcwhorter` and open one result in ParaView.

Day 4 – Single-phase flow: the Theis well test

Week 1 · 6 h

Goal. Understand every block of a simple OGS project file by changing one input at a time in the Theis benchmark and predicting the result.

Block	Hours	What you do
Reading	1.0	Guide 3.2 (anatomy of a .prj); Theis solution
Hands-on	3.0	Read axisym_theis.prj, change permeability and storage, rerun
Testing	1.0	Compare drawdown with the analytical curve
Notes	1.0	One sentence per XML block in your own words
Total	6.0	

> **WHY THIS MATTERS**
> The Theis solution describes how pressure spreads from a well pumping at constant rate in a uniform layer. Your hydrogen well is the same geometry. If you understand why pressure change grows with log(time) here, you will understand pressure build-up during H2 injection.

Steps

1. Open the file in VS Code and identify the blocks `<meshes>`, `<processes>`, `<media>`, `<time_loop>`, `<parameters>`, `<process_variables>`, `<linear_solvers>`:

```
cd ~/work/h2-chert-storage-ogs/starter/02_benchmarks/ogs_src/Tests/Data
cd Parabolic/LiquidFlow/AxiSymTheis
code axisym_theis.prj
```

2. Note the attribute `axially_symmetric="true"` on the mesh: the 2D mesh is a slice (r, z) that OGS rotates around the axis.
3. Make a copy, double the `permeability` value in the copy, and run it. Prediction first: drawdown should roughly halve **[Tested by Claude on Linux]**:

```
cp axisym_theis.prj my_theis.prj        # edit my_theis.prj
ogs my_theis.prj -o _out_mine
```

4. Open both results in ParaView, use Filters > Plot Over Line along r, compare pressure profiles.
5. Run the official comparison again to see the analytical curve: `python axisym_theis.py` (needs the venv).

What correct output looks like

OGS prints `OGS completed` and writes .vtu files to _out_mine. With doubled permeability the pressure change near the well is about half of the original.

Common mistakes and fixes

Mistake / symptom	Fix
`Key <...> has been read 0 times` / config error	A typo in an XML tag. OGS reads tags strictly: copy tag names from a working file, never type them from memory.
Results overwritten	Always use `-o <new folder>` for each variant.

Done when

- ☐ Each XML block described in one sentence
- ☐ Permeability experiment done and the prediction checked

Save: my_theis.prj and a screenshot of the two pressure profiles.

> TIP
>
> **If you only have 3 hours:** Read the .prj with Guide 3.2 open and write the one-sentence notes; skip the rerun.

Day 5 – Mechanics: stress around a borehole (Kirsch) — Week 1 · 6 h

Goal. Learn how OGS represents stress, strain and displacement, including its sign convention, using the disc-with-hole benchmark.

Block	Hours	What you do
Reading	1.5	Kirsch solution; OGS conventions page (tensor order [xx, yy, zz, xy])
Hands-on	2.5	Run kirsch, look at sigma components in ParaView
Testing	1.0	Check hoop stress at the hole = 3x far-field (uniaxial case)
Notes	1.0	Sign convention summary
Total	6.0	

> **WHY THIS MATTERS**
> Your fault analysis is built on stresses. OGS writes tensors as [xx, yy, zz, xy] for 2D and uses tension-positive signs; geomechanics papers usually use compression-positive. Mixing these up flips the sign of the Coulomb stress change, so learn it here on a case with a known answer.

Steps

1. Run and look at the figures **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/02_benchmarks && export PYVISTA_OFF_SCREEN=true
python run_benchmarks.py kirsch
```

2. Open `ogs_src/Tests/Data/Mechanics/Linear/DiscWithHole/_out/*.pvd` in ParaView and colour by `sigma` component 0 (xx), 1 (yy), 2 (zz), 3 (xy).
3. Read the official convention (tension positive, 2D order xx, yy, zz, xy). It is also verified in starter/04_fault_section/test_coulomb.py test 5.

What correct output looks like

PASS for kirsch. In ParaView the stress concentrates at the hole edge.

Common mistakes and fixes

Mistake / symptom	Fix
Using component 2 as shear	In 2D, component 2 is sigma_zz (out of plane); shear xy is component 3.

Done when

- ☐ Can explain tension-positive vs compression-positive
- ☐ Know the 2D tensor order

Save: notes/day05.md with the sign convention in your own words.

> TIP
> **If you only have 3 hours:** Run kirsch and read the conventions page; skip ParaView.

Day 6 – Hydro-mechanics: effective stress and consolidation

Week 1 · 6 h

Goal. Run the confined-compression hydro-mechanics benchmark and understand the Biot coefficient and effective stress.

Block	Hours	What you do
Reading	1.5	Effective stress (Terzaghi/Biot), Bishop stress for two phases (Guide 3.6)
Hands-on	2.5	Run confined_compression, inspect pressure and displacement over time
Testing	1.0	Change biot_coefficient in a copy and predict the change
Notes	1.0	Draw the coupling diagram (Figure 3) by hand
Total	6.0	

> **WHY THIS MATTERS**
> Pore pressure carries part of the load. Effective stress = total stress + Biot x pressure (tension positive). When you inject H2, pressure rises, effective compression falls, and a fault comes closer to slipping. That is the whole mechanism behind your Coulomb analysis.

Steps

1. Run **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/02_benchmarks
python run_benchmarks.py confined_compression
```

2. Open `ogs_src/Tests/Data/HydroMechanics/Linear/Confined_Compression/square_1e2.prj` and find `biot_coefficient`, the `HYDRO_MECHANICS` process and the displacement boundary conditions.
3. Copy the .prj, change biot_coefficient from its value to half of it, run with `ogs <copy>.prj -o _out_biot`, and compare displacement.

What correct output looks like

PASS. Lower Biot coefficient gives smaller pressure-driven deformation.

Common mistakes and fixes

Mistake / symptom	Fix
Running the copy in the same output folder	Use `-o _out_biot`.

Done when

- ☐ Can state the effective-stress equation used by OGS

☐ Biot experiment done

Save: Copy of the modified .prj and your hand-drawn coupling diagram (photo).

> TIP
> **If you only have 3 hours:** Run the benchmark and read the effective-stress section only.

Day 7 – Meshing with gmsh and ogstools (Milestone 1)

Week 1 · MILESTONE

Goal. Build the radial mesh of the study yourself, understand physical groups and boundary meshes, and pass the week-1 milestone.

Block	Hours	What you do
Reading	1.0	Guide 3.3 (meshes); make_mesh.py line by line
Hands-on	2.5	Generate meshes with different cell counts; view in ParaView
Testing	1.0	Milestone check (below)
Notes	1.5	Week-1 summary: what works, what is unclear
Total	6.0	

> **WHY THIS MATTERS**
> The mesh decides accuracy and run time. Small cells near the well catch steep pressure changes; large cells far away save time. OGS needs separate boundary meshes (with `bulk_node_ids`) to know where boundary conditions act; ogstools creates them from gmsh physical groups.

Steps

1. Generate the default mesh and a finer one **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2
python make_mesh.py
python make_mesh.py --nr 120 --nz 20 --out mesh_fine
```

2. Open `mesh/domain.vtu` and `mesh/well.vtu` in ParaView (Surface With Edges).
3. Read make_mesh.py: find where the physical groups `domain`, `well`, `outer`, `top`, `bottom` are created, and where `bulk_node_ids` are added to the domain.
4. **Milestone check** - answer in your notes without looking: what is a .prj, a .vtu, a .pvd; what does `-o` do; what is the venv; what is a boundary mesh.

What correct output looks like

- `meshes written: ['bottom', 'domain', 'outer', 'top', 'well'] cells in domain: 600` for the default.
- mesh_fine has 2400 cells.

Common mistakes and fixes

Mistake / symptom	Fix
Domain has more cells than nr x nz	Line cells were mixed in. make_mesh.py uses `Meshes.from_gmsh(..., dim=2)` to keep only 2D cells in the domain.
Mesh files missing for OGS	OGS looks for mesh paths relative to the .prj; keep the mesh/ folder next to the .prj.

Done when

- ☐ Default and fine meshes generated and viewed
- ☐ All six benchmarks still PASS
- ☐ Milestone questions answered
- ☐ Week-1 summary written

Save: notes/week1.md and a screenshot of the radial mesh.

> TIP
> **If you only have 3 hours:** Generate the default mesh, view it, and answer the milestone questions.

Day 8 – Time stepping, convergence and the log file

Week 2 · 6 h

Goal. Run a one-cycle version of the radial H2 model and learn to read the OGS log: time steps, Newton iterations, rejected steps.

Block	Hours	What you do
Reading	1.0	Guide 3.4; time_loop block of radial_drainage.prj
Hands-on	2.5	Build and run the 1-cycle test; change maximum_dt
Testing	1.5	Compare step counts and results
Notes	1.0	Rules of thumb for time stepping
Total	6.0	

> **WHY THIS MATTERS**
> Two-phase flow is strongly non-linear: permeability depends on saturation, saturation on pressure. OGS solves each time step with Newton iterations. IterationNumberBasedTimeStepping grows the step when few iterations are needed and shrinks it when many are needed. A rejected step means Newton failed and OGS retries with a smaller step.

Steps

1. Build and run one storage cycle (about 2-3 min) **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2
python build_prj.py --cycles 1 --tag test
ogs radial_drainage_test.prj -o out_test > out_test.log 2>&1
grep -E 'accepted steps|rejected' out_test.log | head -2
```

2. Look inside the log for one time step: `grep -m 20 -A3 'Time step #10' out_test.log`.
3. Edit `maximum_dt` in build_prj.py (search `maximum_dt`) from 2 days to 0.5 days, rebuild with `--tag dt05`, run, compare step count and run time.

What correct output looks like

About 280 accepted steps and 0 rejected for the 1-cycle test on a 4-core machine; `OGS completed` at the end.

Common mistakes and fixes

Mistake / symptom	Fix
Many rejected steps	Reduce `maximum_dt` or `initial_dt`; check that schedule switches are output times (they are, by construction in schedule.py).
`Mismatch in numerical Jacobian perturbation configuration`	The `<epsilons>` list must have one value per non-displacement variable (3 for TH2M: p_g, p_c, T).

Done when

- ☐ 1-cycle run finished
- ☐ Can explain accepted vs rejected steps
- ☐ dt experiment compared

Save: out_test.log and a table of max_dt vs steps vs run time.

> TIP
>
> **If you only have 3 hours:** Run the 1-cycle test and read the summary lines only.

Day 9 – Two-phase flow theory and the McWhorter benchmark

Week 2 · 6 h

Goal. Understand capillary pressure, saturation and relative permeability curves, and reproduce the McWhorter-Sunada analytical solution.

Block	Hours	What you do
Reading	2.0	McWhorter & Sunada (1990); Guide 1.3 and glossary entries
Hands-on	2.0	Plot van Genuchten curves with props.py; rerun mcwhorter
Testing	1.0	Read the error plot; locate the largest error
Notes	1.0	Explain each curve in two sentences
Total	6.0	

> **WHY THIS MATTERS**
> Capillary pressure (gas pressure minus liquid pressure) controls how much brine stays in the small pores of the chert. Relative permeability controls how easily each phase flows when both are present. These curves decide how much H2 you can push in and get back.

Steps

1. Plot the curves used in the study (van Genuchten with the values in properties.csv) **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/common
python - <<'EOF'
import numpy as np, props
P = props.load()
for S in (0.5, 0.6, 0.7, 0.9):
    pc = props.vg_pc(S, P['res_vG_pb'], P['res_vG_m'], P['res_S_Lr'], 0.0)
    print(f'S_L = {S:.2f}  ->  p_c = {pc/1e6:.3f} MPa')
EOF
```

2. Rerun McWhorter and open figures/mcwhorter_0.png **[Tested by Claude on Linux]**:

```
cd ~/work/h2-chert-storage-ogs/starter/02_benchmarks
python run_benchmarks.py mcwhorter
```

What correct output looks like

- S_L = 0.60 gives p_c = 1.764 MPa (the initial state of the radial model).
- McWhorter: numerical and exact saturation agree except at the sharp front, where the relative error reaches about 0.4 (the official check allows < 0.5).

Common mistakes and fixes

Mistake / symptom	Fix
Confusing p_b with entry pressure	In van Genuchten, p_b is a scale pressure, not a sharp entry pressure (see glossary).

Done when

- ☐ Can sketch S_L(p_c) and k_r(S_L)
- ☐ McWhorter PASS and error explained

Save: Your curve printout and the McWhorter figure with one paragraph of interpretation.

> TIP
> **If you only have 3 hours:** Run McWhorter and read the benchmark page.

Day 10 – Verification note 1: McWhorter in your own words

Week 2 · 6 h

Goal. Write a one-page verification note (figure, error measure, conclusion) that becomes part of your paper's supplement.

Block	Hours	What you do
Reading	1.0	Grunwald et al. (2022) verification section
Hands-on	2.5	Extract numbers from the McWhorter output with ogstools
Testing	1.0	Check your numbers against the official script's error
Notes	1.5	Write the note
Total	6.0	

> **WHY THIS MATTERS**
> Reviewers trust a model when they see it reproduce known solutions with a stated error. A short, honest verification note (including where the error is largest and why) is the credibility base of your paper.

Steps

1. Read the result with ogstools [Tested by Claude on Linux]:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/02_benchmarks/ogs_src/Tests/Data/TH2M/H2/mcWhorter
python - <<'EOF'
import ogstools as ot
ms = ot.MeshSeries('_out/result_McWhorter_H2.pvd')
print(len(ms), 'outputs, last time', ms.timevalues[-1], 's')
print('saturation range', ms[-1].point_data['saturation'].min(),
      ms[-1].point_data['saturation'].max())
EOF
```

2. Write notes/verification_mcwhorter.md: problem, parameters (from the .prj), figure, maximum error and where, conclusion.

What correct output looks like

The script prints the number of outputs and a saturation range of roughly 0.05-0.8.

Common mistakes and fixes

Mistake / symptom	Fix
Claiming 'excellent agreement' without a number	Always give the error measure and where it is largest.

Done when

☐ Verification note written with a figure and an error number

Save: notes/verification_mcwhorter.md

> TIP
> **If you only have 3 hours:** Write the note using the official figure only.

Day 11 – Two-phase flow with mechanics: Liakopoulos

Week 2 · 6 h

Goal. Run the Liakopoulos drainage experiment benchmark (TH2M with mechanics) and compare with the measured data.

Block	Hours	What you do
Reading	1.5	Benchmark page and the experiment (Liakopoulos 1965, as used by OGS)
Hands-on	2.0	Run, read the comparison figures
Testing	1.0	Identify which variables match the experiment
Notes	1.5	Verification note 2
Total	6.0	

> **WHY THIS MATTERS**
> This is the closest official test to your coupled problem: gas and water flow plus deformation in the same TH2M process. It checks that the hydro-mechanical coupling behaves physically before you trust it on a reservoir.

Steps

1. Run **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/02_benchmarks && export PYVISTA_OFF_SCREEN=true
python run_benchmarks.py liakopoulos
```

2. Open figures/liakopoulos_*.png and the .prj; find `TH2M`, `bishops_effective_stress`, the saturation and relative permeability models.
3. Write notes/verification_liakopoulos.md.

What correct output looks like

PASS; figures compare OGS with experimental/reference data.

Common mistakes and fixes

Mistake / symptom	Fix
Running from the wrong folder	Always `cd starter/02_benchmarks` first; the runner finds the files itself.

Done when

- ☐ Liakopoulos PASS
- ☐ Verification note 2 written

Save: notes/verification_liakopoulos.md

TIP

If you only have 3 hours: Run it and save the figures; write the note tomorrow morning.

Day 12 – Hydrogen and gas appearance: Bourgeat benchmark

Week 2 · 6 h

Goal. Reproduce the published hydrogen-injection benchmark with gas phase appearance and compare with the reference curves.

Block	Hours	What you do
Reading	1.5	Bourgeat et al. (2013); OGS phase_appearance page
Hands-on	2.0	Run, compare with references/bourgeat_*.csv
Testing	1.0	Read the mean relative error checked by the script
Notes	1.5	Verification note 3; list differences to your reservoir case
Total	6.0	

> **WHY THIS MATTERS**
> Your depleted reservoir has gas, but the caprock and surroundings are brine-saturated. Bourgeat injects H2 into a fully water-saturated rock until a gas phase appears. It tests exactly the dissolution (Henry's law) and phase-appearance machinery your model relies on, using the same H2 component set-up that starter/03_radial_h2 copied.

Steps

1. Run **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/02_benchmarks
python run_benchmarks.py bourgeat
```

2. Open figures/bourgeat_*.png. Note in the .prj how dissolved H2 enters through liquid `density` depending on `concentration`. Your model uses the same construction (DECISIONS.md D7).

What correct output looks like

PASS in about 40 s; gas saturation, gas and liquid pressure match the published curves within the script's tolerance.

Common mistakes and fixes

Mistake / symptom	Fix
Thinking dissolution is automatic	In OGS TH2M dissolved H2 mass is only carried if liquid density depends on `concentration` (tested: without it the dissolved mass was zero).

Done when

- ☐ Bourgeat PASS
- ☐ Verification note 3 written
- ☐ Three verification notes ready for the supplement

Save: notes/verification_bourgeat.md

> TIP
>
> **If you only have 3 hours:** Run it, save figures, write three bullet points.

Day 13 – Property block 1: hydrogen and brine

Week 2 · 6 h

Goal. Compute and document H2 density, viscosity and solubility at reservoir conditions, and understand the error of the density model OGS can use.

Block	Hours	What you do
Reading	1.5	Leachman et al. (2009), Muzny et al. (2013), Sander (2023): what each provides
Hands-on	2.0	compute_h2_properties.py; edit properties.csv sources
Testing	1.0	Check values against the NIST WebBook by hand for 2 pressures
Notes	1.5	Fluid property table with sources
Total	6.0	

> **WHY THIS MATTERS**
> OGS has no built-in hydrogen. Peng-Robinson would be more accurate but fails in TH2M with phase transition (tested, DECISIONS.md D6), so the model uses an ideal gas. You must quantify that error (2.2 % at 4 MPa, 4.5 % at 8 MPa) and state it.

Steps

1. Run the property script **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
python starter/properties/compute_h2_properties.py
```

2. Open starter/properties/properties.csv. For every fluid row, check the `source` column and mark anything you could not confirm as TO-VERIFY.
3. Brine: replace the synthetic brine density and viscosity with values for Kansas Mississippian produced water if you find a citable source **TO-VERIFY**.

What correct output looks like

A table from 2 to 20 MPa: at 8 MPa reference density 5.743 kg/m3, ideal gas 6.002 (+4.5 %), Peng-Robinson 5.847 (+1.8 %); viscosity about 9.5e-6 Pa s.

Common mistakes and fixes

Mistake / symptom	Fix
Using the Muzny correlation coefficients by hand	An erratum exists (2022); use CoolProp, which implements the correlation.
Henry constant units	OGS needs mol/(m3 Pa) (Hcp); 7.8e-6 mol/(m3 Pa) for H2 in pure water at 298 K (Sander 2023).

Done when

- ☐ Fluid rows of properties.csv all have a source or TO-VERIFY
- ☐ Density-model error recorded

Save: properties.csv (committed) and h2_properties_check.txt.

> TIP
> **If you only have 3 hours:** Run the script and copy its table into your notes.

Day 14 – Property block 2: chat petrophysics (Milestone 2) Week 2 · MILESTONE

Goal. Fill the reservoir rows (porosity, permeability, water saturation, residual water) from literature now, and prepare to swap in log-derived values later.

Block	Hours	What you do
Reading	2.0	Watney, Guy & Byrnes (2001); KGS chat posters
Hands-on	2.0	Update properties.csv; prepare a log-to-CSV mapping
Testing	1.0	Milestone check
Notes	1.0	Week-2 summary
Total	6.0	

> **WHY THIS MATTERS**
> The chat is high-porosity but low-permeability, and microporous chert holds a lot of immobile water. In the model this enters only through effective properties (limit 3): residual liquid saturation, upscaled permeability, capillary curve. Every value needs a source tag so your later log-derived values can replace them without touching the models.

Steps

1. Review the reservoir rows of properties.csv (res_porosity, res_permeability, S_L_init, res_S_Lr, res_thickness, z_top_res).
2. Write a mapping table in notes: which output of your log-based neural-field study will replace which key (for example mean porosity of the net interval -> res_porosity) and how you will upscale (arithmetic mean for porosity, geometric or harmonic for permeability).
3. Run a quick model check that nothing breaks after edits **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2
python build_prj.py --cycles 1 --tag check
```

4. **Milestone check**: three verification notes done; properties.csv has no empty source; you can explain the three limits.

What correct output looks like

`wrote radial_drainage_check.prj: ...` without errors.

Common mistakes and fixes

Mistake / symptom	Fix
Copying a porosity from a different formation	Keep only chat values; if unsure, keep the synthetic value and tag it TO-VERIFY.

Done when

- ☐ Reservoir rows sourced
- ☐ Log mapping table written
- ☐ Milestone 2 passed
- ☐ Week-2 summary written

Save: properties.csv and notes/week2.md

> TIP
> **If you only have 3 hours:** Fill porosity and permeability sources only and do the milestone check.

Day 15 – Property block 3: capillary, relative permeability and trapping

Week 3 · 6 h

Goal. Choose and justify the capillary and relative-permeability curves and compute the Land trapped-gas saturation for the imbibition bracket.

Block	Hours	What you do
Reading	2.0	Land (1968); Lysyy et al. (2022) on H2 hysteresis
Hands-on	2.0	Compute S_gr for C = 1, 2, 5; plot curves
Testing	1.0	Check that S_L_init stays inside both brackets' curves
Notes	1.0	Paragraph: why a bracket and not a prediction
Total	6.0	

> **WHY THIS MATTERS**
> OGS has no hysteresis (limit 1). Real H2 cycling shows strong relative-permeability hysteresis (Lysyy et al. 2022), meaning some gas is trapped when brine re-enters. The bracket gives an upper bound (no trapping) and a lower bound (maximum Land trapping everywhere). The truth lies in between.

Steps

1. Compute trapped gas for several Land coefficients **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/common
python -c "import props; P=props.load(); Sgi=1-P['res_S_Lr']; \
[print(C, round(props.land_trapped(Sgi, C), 3)) for C in (1, 2, 5)]"
```

2. Note the base value used by build_prj.py: C = 2 gives S_gr = 0.262.

What correct output looks like

C = 1 -> 0.355, C = 2 -> 0.262, C = 5 -> 0.147.

Common mistakes and fixes

Mistake / symptom	Fix
Using S_gr above the initial gas saturation (0.40)	Then the native gas would be immobile from the start; keep S_gr < 1 - S_L_init.

Done when

- ☐ Curve choice justified in notes
- ☐ Bracket values computed

Save: notes/day15_curves.md

TIP

If you only have 3 hours: Compute the three S_gr values and write the paragraph.

Day 16 – Property block 4: rock mechanics and stress state

Week 3 · 6 h

Goal. Document the elastic properties, the in-situ stress state and the fault friction, with sources and sensitivity ranges.

Block	Hours	What you do
Reading	2.0	Alt & Zoback (2017); Byerlee (1978); Rubinstein et al. (2018) for the seismicity context
Hands-on	1.5	Update stress rows; list ranges
Testing	1.5	Run test_coulomb.py and understand each test
Notes	1.0	Stress-state table
Total	6.0	

> **WHY THIS MATTERS**
> Whether a fault slips depends on the background stress more than on the injection. Alt & Zoback (2017) report SHmax about N85E and strike-slip/normal faulting in northern Oklahoma and southern Kansas; the starter uses that, but extrapolating to Barber County is TO-VERIFY. Friction 0.6 is Byerlee's typical value.

Steps

1. Run the Coulomb verification **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/04_fault_section
python test_coulomb.py
```

2. Read each test in test_coulomb.py and write in your notes what physical fact it checks.
3. Update Sv_grad, Shmin_ratio, SHmax_ratio, SHmax_azimuth, friction if you find better sources (World Stress Map, KGS reports) **TO-VERIFY**.

What correct output looks like

Six PASS lines and `ALL PASS`, including `optimal dip = 45 + atan(mu)/2 = 60.48 (got 60.48)`.

Common mistakes and fixes

Mistake / symptom	Fix
Mixing compression-positive and tension-positive	Library functions in coulomb.py are compression-positive; OGS output is converted by ogs2d_to_tensor().

Done when

- ☐ Mechanical and stress rows sourced or TO-VERIFY
- ☐ test_coulomb.py ALL PASS

☐ Stress table written

Save: notes/day16_stress.md

> **TIP**
> **If you only have 3 hours:** Run test_coulomb.py and read Alt & Zoback's abstract.

Day 17 – Build the radial (well) model

Week 3 · 6 h

Goal. Understand every line of build_prj.py for the radial model and reproduce the expected one-cycle mass balance.

Block	Hours	What you do
Reading	1.5	Guide 3.7.1; build_prj.py; schedule.py
Hands-on	2.5	Build, run 1 cycle, post-process
Testing	1.0	Mass balance < 1 %
Notes	1.0	Diagram of geometry and boundary conditions
Total	6.0	

> **WHY THIS MATTERS**
> A 2D axisymmetric model is a cheap way to represent one well in a round, closed compartment. The mass-balance check (H2 in the model vs H2 injected by the schedule) is your first self-test: if it fails, boundary-flux scaling or units are wrong.

Steps

1. Build, run and post-process one cycle **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2
python make_mesh.py
python build_prj.py --cycles 1 --tag c1
ogs radial_drainage_c1.prj -o out_c1 > out_c1.log 2>&1
python postprocess.py out_c1/radial_drainage_c1.pvd --prj radial_drainage_c1.prj
```

2. Open out_c1/radial_drainage_c1_inventory.png and out_c1/*.pvd in ParaView (colour by saturation; watch the H2 plume grow and shrink).

What correct output looks like

- `mass-balance error before first withdrawal:` below 1 %.
- One line `cycle 1: injected 1037 t, produced ... t, RF = ...` (RF about 0.7 for the drainage bracket).

Common mistakes and fixes

Mistake / symptom	Fix
`The required bulk node ids map does not exist in the boundary mesh 'domain'`	The domain mesh is used as a boundary for temperature; make_mesh.py adds bulk ids. Regenerate the mesh with make_mesh.py.
Dissolved H2 is zero	Liquid density must depend on `concentration` (already in build_prj.py).

Done when

- ☐ 1-cycle run done
- ☐ Mass balance < 1 %
- ☐ Geometry/BC diagram drawn

Save: out_c1/*_summary.csv, *_inventory.png

> TIP
> **If you only have 3 hours:** Run the four commands and save the inventory figure.

Day 18 – Radial model: full cycles and both trapping brackets

Week 3 · 6 h

Goal. Run three seasonal cycles for both trapping brackets and reproduce the expected recovery factors.

Block	Hours	What you do
Reading	1.0	Guide 1.3 (bracket) and 3.7.1 (recovery definition)
Hands-on	2.5	Run both brackets in parallel (5-8 min each)
Testing	1.5	Compare with expected/; optional gravity run
Notes	1.0	Interpretation paragraph
Total	6.0	

> **WHY THIS MATTERS**
> Recovery factor here = H2 produced during a withdrawal period divided by H2 injected in the preceding injection period. The difference between the two brackets is the uncertainty caused by trapping, which OGS cannot resolve directly. Reporting both is honest.

Steps

1. Build and run both brackets at the same time (2 processes) [Tested by Claude on Linux]:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2
python build_prj.py --bracket drainage
python build_prj.py --bracket imbibition
mkdir -p out
ogs radial_drainage.prj -o out > out/radial_drainage.log 2>&1 &
ogs radial_imbibition.prj -o out > out/radial_imbibition.log 2>&1 &
wait
```

2. Post-process both [Tested by Claude on Linux]:

```
python postprocess.py out/radial_drainage.pvd
python postprocess.py out/radial_imbibition.pvd
```

3. Optional: switch gravity on (1 cycle) and compare [Tested by Claude on Linux]:

```
python build_prj.py --cycles 1 --gravity --tag gravity
mkdir -p out_gravity
ogs radial_drainage_gravity.prj -o out_gravity > out_gravity/log.txt 2>&1
```

What correct output looks like

- Drainage bracket RF: 0.71, 0.79, 0.85 (cycles 1-3); imbibition bracket: 0.40, 0.39, 0.38; mass-balance error < 1 %.
- Run time about 5 min (drainage) and 8 min (imbibition) on 4 cores.

Common mistakes and fixes

Mistake / symptom	Fix
Comparing runs with different properties.csv	Commit properties.csv before each batch and note the commit hash with the results.

Done when

- ☐ Both brackets run and post-processed
- ☐ Numbers match expected/ within a few %
- ☐ Interpretation written

Save: out/*_summary.csv, *_inventory.png (copy to results/)

> TIP
> **If you only have 3 hours:** Run only the drainage bracket.

Day 19 – Build the fault cross-section

Week 3 · 6 h

Goal. Generate the 2D plane-strain section with layers and a fault band, start the 20-minute run, and understand the superposition approach.

Block	Hours	What you do
Reading	1.5	Guide 3.7.2; make_mesh.py and build_prj.py of 04_fault_section
Hands-on	2.0	Mesh, inspect, build, start run
Testing	1.5	Check material IDs and run statistics
Notes	1.0	Section sketch with materials and BCs
Total	6.0	

> **WHY THIS MATTERS**
> A fault band is a thin strip of ordinary elements with its own properties. It cannot slip (limit of this approach), but it tells you how stress changes on the fault plane. Stress changes are computed from the depleted initial state; the in-situ stress is added afterwards (superposition, valid for linear elasticity).

Steps

1. Mesh and build **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/04_fault_section
python make_mesh.py
python build_prj.py
```

2. Start the run (about 19 min on 4 cores) and read Guide 3.7.2 while it runs **[Tested by Claude on Linux]**:

```
mkdir -p out
ogs section_drainage.prj -o out > out/section_drainage.log 2>&1
```

3. Open mesh/domain.vtu in ParaView, colour by MaterialIDs (0 overburden, 1 caprock, 2 reservoir, 3 underburden, 4 fault).

What correct output looks like

- `cells: 9372 | cells per MaterialID: {0: 3320, 1: 1620, 2: 710, 3: 3316, 4: 406}`
- Log ends with 187 accepted steps and 0 rejected.

Common mistakes and fixes

Mistake / symptom	Fix
`Newton: The linear solver failed` at step 1	Happens if seals start with no gas phase at all (negative p_c). Use the uniform initial p_c (capillary equilibrium) as in build_prj.py (DECISIONS.md D12).

Done when

- ☐ Mesh checked
- ☐ Run finished with 0 rejected steps

Save: out/section_drainage.log and a mesh screenshot.

> TIP
>
> **If you only have 3 hours:** Mesh and build; start the run overnight.

Day 20 – Coulomb stress on the fault

Week 3 · 6 h

Goal. Compute the Coulomb stress change along the fault, interpret the base case and the sensitivity table.

Block	Hours	What you do
Reading	1.5	King et al. (1994) on Coulomb stress; Guide 1.5
Hands-on	2.0	coulomb.py; inspect CSV outputs
Testing	1.5	Check sigma - sigma_total = Bishop pressure in the output
Notes	1.0	Interpretation including what is NOT predicted
Total	6.0	

> **WHY THIS MATTERS**
> Coulomb failure function CFF = shear stress - friction x effective normal stress. A positive change (dCFS > 0) moves the fault toward failure. A fault already close to failure (CFF near 0) can be pushed over by small changes; that is why the background stress state matters so much.

Steps

1. Run the analysis [Tested by Claude on Linux]:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/04_fault_section
python coulomb.py out/section_drainage.pvd
```

2. Check the stress convention in your own output [Tested by Claude on Linux]:

```
python - <<'EOF'
import glob, numpy as np, pyvista as pv
m = pv.read(sorted(glob.glob('out/section_drainage_t_*.vtu'))[5])
peff = m['gas_pressure'] - m['saturation'] * m['capillary_pressure']
d = m['sigma'] - m['sigma_total']
print('max |diag - p_eff| [Pa]:', np.abs(d[:, 0] - peff).max())
EOF
```

What correct output looks like

- `max dCFS = 0.856 MPa at t = 360 d on 72 fault cells`; base case CFF0_max = -0.29 MPa, after = +0.55 MPa.
- The convention check prints a few hundred Pa or less.

Common mistakes and fixes

Mistake / symptom	Fix
Reading CFF_after > 0 as 'the fault will slip'	This is a screening indicator with uncalibrated stress; it says the stress state must be calibrated before conclusions.
MaterialIDs missing in output	coulomb.py reads them from mesh/domain.vtu (same cell order).

Done when

- ☐ Coulomb CSVs and figure produced
- ☐ Convention check done
- ☐ Interpretation written with limits

Save: out/section_drainage_coulomb*.csv and .png

> TIP
> **If you only have 3 hours:** Run coulomb.py and read the sensitivity table.

Day 21 – Experiment design and cushion-gas study (Milestone 3)

Week 3 · MILESTONE

Goal. Design the run matrix for the paper and run the cushion-gas series.

Block	Hours	What you do
Reading	1.0	Guide 3.7.1 (cushion definition); your research questions
Hands-on	2.5	Write run_matrix.csv; run the cushion series
Testing	1.5	Milestone check
Notes	1.0	Week-3 summary
Total	6.0	

> **WHY THIS MATTERS**
> Cushion gas is the H2 left in place to keep pressure up (here it is H2, not N2 or CH4: limit 2). More cushion usually raises recovery per cycle but locks up more H2. Varying the cushion-filling time changes the cushion fraction = cushion mass / (cushion + working mass).

Steps

1. Run a cushion series with the `--set` override and a tag. Example loop (each run about 5 min) **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2 && mkdir -p out_tc
for tc in 90 180 360; do
  python build_prj.py --set t_cushion=$tc --tag tc$tc
  ogs radial_drainage_tc$tc.prj -o out_tc > out_tc/tc$tc.log 2>&1
  python postprocess.py out_tc/radial_drainage_tc$tc.pvd \
      --prj radial_drainage_tc$tc.prj
done
```

2. Collect cushion_fraction and recovery_factor from out_tc/*_summary.csv into one table.
3. **Milestone check**: both models run; mass balance < 1 %; Coulomb tests pass; run matrix written.

What correct output looks like

- Real results (3 cycles, drainage bracket): cushion 90 d -> fraction 0.43, RF 0.48 / 0.63 / 0.74; 180 d -> 0.60, RF 0.71 / 0.79 / 0.85; 360 d -> 0.75, RF 1.19 / 1.11 / 1.06. Mass balance about 1 % in all runs.
- RF above 1 is not an error: with a large cushion the reservoir pressure stays well above the withdrawal limit (BHP = initial pressure), so withdrawal also produces part of the cushion.

Common mistakes and fixes

Mistake / symptom	Fix
Reporting per-cycle RF > 1 as “better than 100 %”	Also report cumulative recovery and the change of H2 in place after each cycle (H2_in_place_t in *_inventory.csv), or raise p_bhp_min (`--set p_bhp_min=...`) so the cushion is protected. Record your choice in DECISIONS.md.
Overwriting .prj files	Always use a unique `--tag`.

Done when

- ☐ Run matrix written
- ☐ Cushion series done
- ☐ Milestone 3 passed

Save: results/cushion_series.csv and notes/week3.md

> TIP
>
> **If you only have 3 hours:** Run two cushion values (90 and 360 days).

Day 22 – Trapping bracket sensitivity

Week 4 · 6 h

Goal. Quantify how strongly recovery depends on the assumed trapping (Land coefficient C) and report it as a bracket.

Block	Hours	What you do
Reading	1.0	Lysyy et al. (2022) results
Hands-on	3.0	Imbibition runs for C = 1 and 5
Testing	1.0	Check trend vs C = 2 result
Notes	1.0	Bracket figure and wording
Total	6.0	

> **WHY THIS MATTERS**
> The imbibition bracket assumes maximum trapping everywhere, which is pessimistic. Showing how the bound moves with C tells the reader how much of the uncertainty comes from this single unknown parameter.

Steps

1. Run [Tested by Claude on Linux]:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2 && mkdir -p out_C
for C in 1 5; do
  python build_prj.py --bracket imbibition --set land_C=$C --tag C$C
  ogs radial_imbibition_C$C.prj -o out_C > out_C/C$C.log 2>&1
  python postprocess.py out_C/radial_imbibition_C$C.pvd \
      --prj radial_imbibition_C$C.prj
done
```

2. Plot RF vs cycle for drainage, C = 5, C = 2, C = 1.

What correct output looks like

- Real results (3 cycles): C = 1 (S_gr 0.355): RF 0.23 / 0.18 / 0.12; C = 2 (S_gr 0.262): 0.40 / 0.39 / 0.38; C = 5 (S_gr 0.147): 0.61 / 0.67 / 0.72; drainage bracket: 0.71 / 0.79 / 0.85.
- So the trapping assumption alone moves cycle-3 recovery between 0.12 and 0.85: the single largest uncertainty of the study.

Common mistakes and fixes

Mistake / symptom	Fix
Calling the bracket a confidence interval	It is a bound from two model assumptions, not a statistical interval.

Done when

- ☐ C = 1 and C = 5 runs post-processed
- ☐ Bracket figure drafted

Save: results/trapping_bracket.csv and figure

> TIP
> **If you only have 3 hours:** Run C = 5 only.

Day 23 – Sensitivity: permeability, micro-porosity proxy, capillarity, rate

Week 4 · 6 h

Goal. Run a one-at-a-time sensitivity study on the main reservoir parameters and make a tornado chart of recovery factor.

Block	Hours	What you do
Reading	0.5	One-at-a-time sensitivity: what it can and cannot tell
Hands-on	3.5	Eight runs (low/high of four parameters), 1 cycle each
Testing	1.0	Check each run's mass balance
Notes	1.0	Tornado chart
Total	6.0	

> **WHY THIS MATTERS**
> Micro-porosity cannot be modelled explicitly (limit 3); its proxy is the residual liquid saturation res_S_Lr (immobile water). Sensitivity shows which parameter you most need to pin down with your log data.

Steps

1. Run low/high values with 1 cycle (about 2.5 min each) **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2 && mkdir -p out_sens
for s in res_permeability=3e-15 res_permeability=3e-14 res_S_Lr=0.35 res_S_Lr=0.5 \
         res_vG_pb=2e5 res_vG_pb=1e6 rate_inj=0.05 rate_inj=0.2; do
  tag=$(echo $s | tr '=.' '__')
  python build_prj.py --cycles 1 --set $s --tag $tag
  ogs radial_drainage_$tag.prj -o out_sens > out_sens/$tag.log 2>&1
  python postprocess.py out_sens/radial_drainage_$tag.pvd \
      --prj radial_drainage_$tag.prj
done
```

2. Collect RF and pg_max_MPa per run; draw a tornado chart (bars: RF change from base).

What correct output looks like

- Real 1-cycle RF (base 0.707): permeability 3e-15 -> 0.26, 3e-14 -> 1.50 (cushion gas produced; mass balance 2.2 %); res_S_Lr 0.35 -> 0.60, 0.50 -> 0.76; p_b 2e5 -> 0.70, 1e6 -> 0.71; rate 0.05 -> 0.66, 0.2 -> 0.77 kg/s.
- Permeability dominates, the capillary scale pressure barely matters in this set-up. Runs with res_S_Lr = 0.5 need S_L_init > 0.5 (it is 0.60, fine).

Common mistakes and fixes

Mistake / symptom	Fix
res_S_Lr above S_L_init	build_prj.py will produce an invalid initial capillary pressure; keep res_S_Lr < S_L_init.
A run fails to converge	Reduce rate or maximum_dt for that run and note it in DECISIONS.md.

Done when

- ☐ Eight runs done
- ☐ Tornado chart drafted

Save: results/sensitivity_radial.csv and tornado figure

> **TIP**
> **If you only have 3 hours:** Run permeability and S_Lr pairs only (4 runs).

Day 24 – Fault sensitivity and the safe operating window

Week 4 · 6 h

Goal. Re-run the section with other fault dips and rates, and define the safe operating window from pressure and Coulomb criteria.

Block	Hours	What you do
Reading	1.0	Guide 1.5 and 3.7.2
Hands-on	3.0	Re-mesh dip 45 and 75 (2 x 20 min); rate variants
Testing	1.0	Compare re-meshed results with the plane-rotation approximation
Notes	1.0	Safe-window criteria and table
Total	6.0	

> **WHY THIS MATTERS**
> The safe operating window is the range of injection rate and pressure where (a) the fault stays below failure (CFF0 + dCFS < 0) and (b) gas pressure stays well below the minimum horizontal stress (no tensile fracturing of the caprock). Both depend on uncertain stress, so report the window for each stress scenario.

Steps

1. Full re-mesh for another dip (repeat for 75) **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/04_fault_section
python make_mesh.py --dip 45 --out mesh
python build_prj.py --tag dip45
ogs section_drainage_dip45.prj -o out_dip45 > out_dip45.log 2>&1
python coulomb.py out_dip45/section_drainage_dip45.pvd --set fault_dip=45
```

2. Regenerate the base mesh afterwards: `python make_mesh.py`.
3. Rate series: `python build_prj.py --set line_rate=2e-4 --tag q2` and run, then coulomb.py.
4. Define criteria in notes (example): max gas pressure < 0.9 x Shmin at reservoir top; max(CFF0 + dCFS) < 0 for the base stress state.

What correct output looks like

- Real result for the re-meshed 45 degree fault: max dCFS = 0.92 MPa on 104 fault cells, CFF0 = -1.18 -> -0.36 MPa (stays below failure); 187 steps, 0 rejected.
- The quick plane-rotation estimate for dip 45 in the base sensitivity table gave 1.23 MPa, about one third too high: use full re-meshing for dip sensitivity in the paper.

Common mistakes and fixes

Mistake / symptom	Fix
Using the plane-rotation shortcut as a result	It over-estimated dCFS by about one third in our test (1.23 vs 0.92 MPa); re-mesh for every dip you report.
Forgetting `--set fault_dip=45` in coulomb.py	The plane normal must match the meshed fault.
Mesh folder left at dip 45	Re-run make_mesh.py without options before other runs.

Done when

- ☐ Dip runs done
- ☐ Safe-window table drafted

Save: results/safe_window.csv

> TIP
>
> **If you only have 3 hours:** Use the plane-rotation sensitivity already in section_drainage_coulomb_sensitivity.csv and draft the criteria.

Day 25 – Mesh and time-step convergence

Week 4 · 6 h

Goal. Show that your key outputs (recovery factor, maximum pressure, dCFS) do not change much when you refine the mesh or the time step.

Block	Hours	What you do
Reading	0.5	Convergence studies in reservoir modelling
Hands-on	3.5	Radial: 3 meshes x 1 cycle; max_dt variants
Testing	1.0	Relative change table
Notes	1.0	Convergence paragraph for the methods
Total	6.0	

> **WHY THIS MATTERS**
> A reviewer will ask whether results depend on the mesh. If refining the mesh changes RF by less than about 2 %, the coarse mesh is good enough for the sensitivity runs. This is required in Q1 journals.

Steps

1. Three radial meshes, 1 cycle each [Tested by Claude on Linux]:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
cd starter/03_radial_h2 && mkdir -p out_conv
for m in "30 5" "60 10" "120 20"; do
  set -- $m
  python make_mesh.py --nr $1 --nz $2
  python build_prj.py --cycles 1 --tag n$1
  ogs radial_drainage_n$1.prj -o out_conv > out_conv/n$1.log 2>&1
  python postprocess.py out_conv/radial_drainage_n$1.pvd \
      --prj radial_drainage_n$1.prj
done
python make_mesh.py      # restore the default mesh
```

2. Time step: change maximum_dt in build_prj.py (2 d -> 0.5 d), rebuild with a tag, compare.

What correct output looks like

- Real 1-cycle RF: 300 cells 0.855, 600 cells 0.707, 2400 cells 0.699. The default 600-cell mesh is within about 1 % of the fine mesh; 300 cells is too coarse.
- The 2400-cell run takes several times longer than the default.

Common mistakes and fixes

Mistake / symptom	Fix
Mesh left fine for later runs	Run `python make_mesh.py` to restore the default.

Done when

- ☐ Mesh convergence table
- ☐ Time-step check
- ☐ Methods paragraph

Save: results/convergence.csv

> TIP
> **If you only have 3 hours:** Compare 60 x 10 and 120 x 20 only.

Day 26 – Analysis and publication-quality figures

Week 4 · 6 h

Goal. Produce the final figure set of the paper from saved CSV results with one script.

Block	Hours	What you do
Reading	0.5	Journal figure guidelines (size, fonts, colour)
Hands-on	4.0	Write results/make_paper_figures.py
Testing	0.5	Regenerate from scratch; check colour-blind safety
Notes	1.0	Figure captions
Total	6.0	

> WHY THIS MATTERS
> Figures made from saved CSVs by a script can be regenerated after every change and checked by reviewers. Use the same colours for the same scenario in every figure.

Steps

1. Start from docs/build/make_figures.py (uses a colour-blind-checked palette) [Tested by Claude on Linux]:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
python docs/build/make_figures.py
```

2. Suggested figure set: (1) conceptual model + mesh; (2) verification panel; (3) H2 plume at end of injection/withdrawal; (4) RF vs cycle with bracket; (5) RF vs cushion fraction; (6) tornado chart; (7) dCFS profile + history; (8) safe operating window.

What correct output looks like

PNG (and PDF/SVG) files in results/figures; every figure has a caption with units and sources.

Common mistakes and fixes

Mistake / symptom	Fix
Two y-axes in one chart	Use two panels instead.
Colour alone distinguishes scenarios	Add line styles or hatching (as in the starter figures).

Done when

- ☐ All 8 figures produced by one script
- ☐ Captions drafted

Save: results/figures/ and make_paper_figures.py

> TIP
> **If you only have 3 hours:** Make figures 4, 6 and 7.

Day 27 – Results tables and the methods section

Week 4 · 6 h

Goal. Write the results tables and a complete first draft of the methods section.

Block	Hours	What you do
Reading	1.0	Methods sections of 2 recent UHS papers
Hands-on	3.5	Tables + methods draft (template in Guide 6.2)
Testing	0.5	Every number in text traced to a CSV
Notes	1.0	List of open questions
Total	6.0	

> **WHY THIS MATTERS**
> The methods section must let another person rebuild your model: software version, equations (process), mesh, properties table with sources, boundary conditions, schedule, verification, post-processing definitions.

Steps

1. Export properties.csv as Table 1 (key, value, unit, tag, source).
2. Write methods using the template in Guide section 6.2; replace every bracket.
3. Check every number in the text against a results CSV.

What correct output looks like

A methods draft of about 1500-2500 words and three tables.

Common mistakes and fixes

Mistake / symptom	Fix
Hiding synthetic values	Keep the tag column in Table 1.

Done when

- ☐ Methods draft complete
- ☐ Tables complete

Save: manuscript/methods.md and tables

> **TIP**
> **If you only have 3 hours:** Table 1 and the model-description paragraph.

Day 28 – Manuscript outline, introduction and limitations (Milestone 4)

Week 4 · MILESTONE

Goal. Write the full manuscript outline, the introduction and the limitations section.

Block	Hours	What you do
Reading	1.0	Heinemann et al. (2021); Ramesh Kumar et al. (2023)
Hands-on	3.0	Outline, introduction, limitations
Testing	1.0	Milestone check
Notes	1.0	Week-4 summary
Total	6.0	

WHY THIS MATTERS

The limitations section is part of your credibility. State the three limits exactly as in Guide 1.7, plus the other model simplifications, and say what each one probably does to the results (over- or under-estimate).

Steps

1. Use the outline in Guide 6.1.
2. Copy the limitation list in Guide 6.3 and add one sentence of expected effect for each.
3. **Milestone check**: outline, introduction, methods, limitations drafted; all figures exist.

What correct output looks like

Manuscript skeleton with intro (about 800-1200 words) and limitations.

Common mistakes and fixes

Mistake / symptom	Fix
Overclaiming site-specific predictions	Use 'Aetna-style screening model' wording throughout.

Done when

- ☐ Outline done
- ☐ Introduction drafted
- ☐ Limitations drafted
- ☐ Milestone 4 passed

Save: manuscript/ folder and notes/week4.md

TIP

If you only have 3 hours: Outline and limitations only.

Day 29 – Reproducibility package

Final days · 6 h

Goal. Make the whole study reproducible from a clean clone, and test that with the container.

Block	Hours	What you do
Reading	0.5	Zenodo/GitHub release basics
Hands-on	3.5	Pin versions, run-all script, container test
Testing	1.5	Rebuild from a clean clone
Notes	0.5	Data-availability statement
Total	6.0	

> **WHY THIS MATTERS**
> Q1 journals increasingly require code and data availability. A container proves the results do not depend on your laptop.

Steps

1. Record versions: `pip freeze > starter/pip_freeze_tested.txt`.
2. Build and test the container (Docker Desktop with WSL2 backend on Windows [Not tested on Windows]; tested by Claude on Linux [Tested by Claude on Linux]):

```
cd ~/work/h2-chert-storage-ogs
docker build -t ogs-h2 -f starter/docker/Dockerfile .
docker run --rm -v "$PWD:/work" -w /work ogs-h2 \
    python starter/00_check_install/check_install.py
```

3. Write run_all.sh calling the steps of Days 17-25 in order; run it once from a fresh clone.
4. Make a GitHub release and archive it on Zenodo for a DOI [Not tested on Windows].

What correct output looks like

check_install.py inside the container ends with `RESULT: ALL OK`.

Common mistakes and fixes

Mistake / symptom	Fix
Container cannot download packages behind a proxy	Pass proxy settings or build on a network without a proxy.

Done when

- ☐ Versions pinned
- ☐ Container test passed
- ☐ run_all.sh works from a clean clone

Save: run_all.sh, Dockerfile, pip_freeze_tested.txt (committed)

TIP

If you only have 3 hours: Pin versions and write run_all.sh; container next week.

Day 30 – Self-review and next steps

Final days · 6 h

Goal. Review the manuscript with the Q1 checklist, fix the biggest gaps, and plan how to bring in the log-derived properties.

Block	Hours	What you do
Reading	1.0	Target journal guide for authors
Hands-on	3.0	Go through the checklist (Guide 6.5); fix top 5 issues
Testing	1.0	Re-run check_install, benchmarks, test_coulomb
Notes	1.0	Next-steps plan
Total	6.0	

> **WHY THIS MATTERS**
> A structured self-review catches most of what reviewers will catch. The biggest next improvement is replacing synthetic properties with log-derived ones from your neural-field study, using properties.csv only.

Steps

1. Final regression check **[Tested by Claude on Linux]**:

```
source ~/ogs-env/bin/activate
cd ~/work/h2-chert-storage-ogs
python starter/00_check_install/check_install.py
cd starter/02_benchmarks && python run_benchmarks.py
cd ../04_fault_section && python test_coulomb.py
```

2. Go through every item of the checklist in Guide 6.5; mark done / not done.
3. Plan: for each TO-VERIFY row in properties.csv, write where the value will come from.

What correct output looks like

All checks pass; checklist filled.

Common mistakes and fixes

Mistake / symptom	Fix
Changing models instead of properties.csv for log data	Only properties.csv should change; models read it.

Done when

- ☐ Checklist completed
- ☐ Top 5 issues fixed
- ☐ Next-steps plan written

Save: manuscript/self_review.md

> **TIP**
> **If you only have 3 hours:** Run the regression check and complete half of the checklist.

5. Risk register

Risk	Signs	Remedy	Seen here?
Non-convergence (Newton fails)	rejected steps, `linear solver failed`	smaller max/initial dt; avoid gas-free zones (uniform p_c); check BC switch times	yes, D12
Phase appearance / disappearance	failure where S_g -> 0	small residual gas in seals via high p_b; keep S_gr < initial S_g	yes
Units error	mass balance off by 2pi, 1000, 86400	everything SI (Pa, s, kg, m); check mass balance every run	checked
Schedule bug	injection during rest	use common/schedule.py; plot the curve	yes, D13
Memory limit	WSL killed, swap	.wslconfig memory; 2D only; fewer outputs	no
Disk full	write errors	delete out*/ folders; fewer output times	no
Wrong stress sign	dCFS sign flips	test_coulomb.py; check sigma - sigma_total = p_eff	checked
Uncalibrated stress	CFF0 > 0 before injection	calibrate with field data; report scenarios	yes
Version drift	results change after update	pin versions; rerun expected/ comparisons	-
Overclaiming	site-specific wording	“Aetna-style screening”; repeat the three limits	-

6. Manuscript skeleton

6.1 Outline

1. Introduction: need for seasonal H2 storage; depleted fields; Midcontinent seismicity (Rubinstein et al. 2018); chat reservoirs (Watney et al. 2001); gap and objective.
2. Geological setting: Mississippian chat, Barber County analogue (literature only), stress regime (Alt & Zoback 2017).
3. Methods: OGS TH2M formulation; verification; models; properties (Table 1 with tags); schedule; trapping bracket; Coulomb analysis; convergence.
4. Results: pressure and plume; recovery vs cycle and cushion; trapping bracket; sensitivity; fault stress; safe operating window.
5. Discussion: what controls recovery in microporous chert; seismicity implications; comparison with sandstone studies; limitations.
6. Conclusions.
7. Data and code availability (repository + Zenodo DOI).

6.2 Methods template

“We used OpenGeoSys 6.5.9 (Bilke et al. 2019; Kolditz et al. 2012) with its TH2M process (Grunwald et al. 2022), run isothermally at [T] °C. TH2M solves mass balances of the gas component (H2) and water in a liquid and a gas phase with equilibrium dissolution (Henry's law, [H] mol m-3 Pa-1; Sander 2023), coupled to linear-elastic deformation through Bishop's effective stress with chi = S_L. H2 density followed the ideal-gas law (error +2 to +6 % vs. Leachman et al. 2009 over [p range]); viscosity was [mu] Pa s (Muzny et al. 2013). The implementation was verified against the McWhorter and Sunada (1990) solution, the Liakopoulos drainage test and the Bourgeat et al. (2013) phase-appearance benchmark (Supplement S1). A 2D axisymmetric model ([N] cells) of a closed compartment of radius [re] m … A 2D plane-strain section with a [w] m fault band … Coulomb stress changes were computed on the fault band as … Trapping was bracketed by … Mesh and time-step convergence … ”

6.3 Limitations (copy into the paper)

1. No relative-permeability or capillary hysteresis in OGS; trapping is reported as a bracket (drainage curve vs. Land-type trapped-gas curve), not predicted.
2. Single gas component: cushion gas is H2 left in place; purity and mixing with CH4 or N2 are not modelled; native residual gas is represented as H2.
3. Effective-continuum chert: micro-pore effects only via immobile water, upscaled permeability and capillary curve; no matrix-fracture transfer.
4. Ideal-gas H2 density (overestimates density by 2-6 % in the operating range).
5. Isothermal; no water evaporation; no geochemical or microbial H2 loss.
6. 2D geometries; closed compartment; fault as an element band without slip; linear elasticity.
7. Synthetic/literature properties and stress state (tags in Table 1); results are screening-level for an Aetna-style setting, not site predictions.

6.4 Reproducibility package

- Repository with starter/, properties.csv (tags + sources), DECISIONS.md, pinned requirements and pip freeze.
- Dockerfile tested from a clean build; run_all.sh; expected/ results for comparison.
- Archived release with DOI; data-availability statement naming OGS version 6.5.9.

6.5 Self-review checklist for a Q1 journal

☐ Novelty stated as a scientific gap, not a software first
☐ Every property has a source or tag; synthetic values clearly labelled
☐ Verification shown (3 benchmarks) with error numbers
☐ Mesh and time-step convergence shown
☐ Mass balance error reported
☐ Trapping bracket explained; no hysteresis claim
☐ Cushion = H2 only; no purity claim
☐ No dual-porosity claim
☐ Stress-state uncertainty shown (scenarios); fault slip not claimed
☐ All figures colour-blind safe, single y-axis, units
☐ Limitations section complete
☐ Code/data availability with DOI
☐ Software versions given
☐ Every number in the text traceable to a CSV

Appendix A. Glossary

Term	Plain-language definition
Axisymmetric	A 2D (r, z) model that represents a 3D body by rotating it around an axis (here the well).
Biot coefficient	Fraction of pore pressure that acts against the rock skeleton (1 = all of it).
Bishop effective stress	Effective stress for two fluids: pressure acting on the solid is p_g - chi p_c, here chi = S_L.
Boundary condition (BC)	What happens at the edge of the model: fixed value (Dirichlet) or fixed flux (Neumann).
Capillary pressure p_c	Gas pressure minus liquid pressure; high in small pores.
Coulomb failure function CFF	tau - mu sigma_n'; failure when it reaches 0 (no cohesion).
Cushion gas	Gas left permanently in the reservoir to keep pressure up; here it is H2.
Depleted reservoir	A gas reservoir whose pressure has dropped after production.
Dirichlet / Neumann	Prescribed value / prescribed flux boundary condition.
Dual porosity	A model with two overlapping pore systems (e.g. fractures and matrix) exchanging fluid; not available for two-phase flow in OGS.
Effective stress	Stress carried by the rock skeleton (total stress corrected for pore pressure).
FEM	Finite element method: the equations are solved on small cells and assembled into a large system.
Henry's law	Dissolved gas concentration = Henry constant x gas partial pressure.
Hysteresis	Curves that differ between drainage and imbibition (history dependence).
Imbibition / drainage	Brine re-entering / brine being pushed out of pores.
Land trapping	Model of how much gas gets trapped when brine re-enters: S_gr = S_gi/(1 + C S_gi).
MaterialIDs	Integer per cell telling OGS which medium (material) the cell belongs to.
Newton iteration	Repeated linearised solve until the non-linear equations are satisfied.
Plane strain	2D model assuming no strain in the third direction (long structures).
Recovery factor	H2 produced / H2 injected (per cycle, in this guide).
Relative permeability	Fraction of the rock's permeability available to one phase when two share the pores.
Residual saturation	Saturation below which a phase cannot flow.
SHmax, Shmin, Sv	Maximum and minimum horizontal stress, vertical stress.
TH2M	OGS process for heat (T), two-phase two-component flow (H2) and mechanics (M).
van Genuchten	Common capillary-pressure/saturation model with parameters p_b and m.
venv	Python virtual environment: an isolated folder of packages.
VTU / PVD	VTK unstructured mesh file / index of a time series of VTU files.
WSL2	Windows Subsystem for Linux, version 2: real Linux inside Windows.

Appendix B. Cheat sheet (verified)

All entries were used in files that ran with OGS 6.5.9 / ogstools 0.8.2 **[Tested by Claude on Linux]**.

Item	Where / example
`ogs file.prj -o out`	run; `-m` mesh dir; `-l` log level; `-p` XML patch; `--version`
`<type>TH2M</type>`	process; variables gas_pressure, capillary_pressure, temperature, displacement
`<mesh axially_symmetric="true">`	axisymmetric meshes
`LinearElasticIsotropic`	solid model; `youngs_modulus`, `poissons_ratio`
`CentralDifferences` + `<epsilons>`	numerical Jacobian; 3 epsilons for TH2M
`AqueousLiquid`, `Gas`, `Solid`	phase types; components named H2 and W
`henry_coefficient`, `diffusion`, `vapour_pressure`, `molar_mass`	component properties
`IdealGasLawBinaryMixture`	gas density in TH2M with phase transition
`Linear` with `liquid_phase_pressure` and `concentration`	brine density (carries dissolved H2)
`SaturationVanGenuchten`	`residual_liquid_saturation`, `residual_gas_saturation`, `p_b`, `exponent`
`RelativePermeabilityVanGenuchten`	liquid k_r; `minimum_relative_permeability_liquid`
`RelativePermeabilityNonWettingPhaseVanGenuchtenMualem`	gas k_r; `min_relative_permeability`
`BishopsPowerLaw`	`bishops_effective_stress`, `exponent` 1 (chi = S_L)
`EffectiveThermalConductivityPorosityMixing`	medium thermal conductivity
`IterationNumberBasedTimeStepping`	`number_iterations`, `multiplier`
`PerComponentDeltaX`	convergence; `abstols` (5 values for 2D TH2M)
`Neumann`, `Dirichlet`, `DirichletWithinTimeInterval`	BCs; `<time_interval><start/><end/>`
`Constant`, `CurveScaled`, `MeshNode`, `Group`	parameter types (Group: by MaterialIDs)
`<medium id="0">`	one medium per MaterialID
secondary variables	`sigma` (effective), `sigma_total`, `saturation`, `gas_density`, `liquid_density`, `porosity`, `velocity_gas`, `mass_fraction_liquid`
`ot.Meshes.from_gmsh(msh, dim=2)`	domain + boundary meshes from gmsh physical groups
`ot.MeshSeries(pvd)`, `.timevalues`, `[i]`	read results

Appendix C. Troubleshooting guide

1. Read the **first** `error` or `critical` line in the log: `grep -n -m3 -E '^(error|critical)' out/run.log`. Later errors are often consequences.
2. Config errors (tag names, counts): compare with a working starter or benchmark file.
3. Convergence errors: reduce `initial_dt` and `maximum_dt`; check initial conditions are consistent (no gas-free zones next to gas zones); check that boundary values switch at output times.
4. Wrong-looking results: check mass balance (postprocess.py), units (SI), signs (tension positive), and the schedule curve.
5. Installation errors: run check_install.py and use the table in 2.8.
6. Still stuck: search the OGS Discourse forum (discourse.opengeosys.org) with the exact error text; post a minimal example.

Appendix D. References

Status: ✔ = bibliographic details checked by Claude with a web search on 11-12 Nov 2026 (title, journal, volume, pages/DOI); ○ = standard reference not individually re-checked (check the DOI before citing).

	Reference
✔	Alt, R.C. II, Zoback, M.D. (2017). In situ stress and active faulting in Oklahoma. Bulletin of the Seismological Society of America 107(1). doi:10.1785/0120160156
○	Bell, I.H., Wronski, J., Quoilin, S., Lemort, V. (2014). Pure and pseudo-pure fluid thermophysical property evaluation and the open-source thermophysical property library CoolProp. Ind. Eng. Chem. Res. 53(6), 2498-2508.
✔	Bilke, L., Flemisch, B., Kalbacher, T., Kolditz, O., Helmig, R., Nagel, T. (2019). Development of open-source porous media simulators: principles and experiences. Transport in Porous Media 130, 337-361. doi:10.1007/s11242-019-01310-1
✔	Bourgeat, A.P., Granet, S., Smaï, F. (2013). Compositional two-phase flow in saturated-unsaturated porous media: benchmarks for phase appearance/disappearance. In: Simulation of Flow in Porous Media, De Gruyter, 1-29. doi:10.1515/9783110282245.81
○	Byerlee, J. (1978). Friction of rocks. Pure and Applied Geophysics 116, 615-626.
✔	Grunwald, N., Maßmann, J., Naumov, D., Kolditz, O., Nagel, T. (2022). Non-isothermal two-phase flow in deformable porous media: systematic open-source implementation and verification procedure. Geomech. Geophys. Geo-Energy Geo-Resour. 8, 107. doi:10.1007/s40948-022-00394-2
✔	Heinemann, N., et al. (2021). Enabling large-scale hydrogen storage in porous media - the scientific challenges. Energy & Environmental Science 14, 853-864. doi:10.1039/D0EE03536J
✔	Hogeweg, S., Strobel, G., Hagemann, B. (2022). Benchmark study for the simulation of Underground Hydrogen Storage operations. Computational Geosciences 26, 1367-1378. doi:10.1007/s10596-022-10163-5
○	King, G.C.P., Stein, R.S., Lin, J. (1994). Static stress changes and the triggering of earthquakes. Bull. Seismol. Soc. Am. 84(3), 935-953.
✔	Kolditz, O., et al. (2012). OpenGeoSys: an open-source initiative for numerical simulation of thermo-hydro-mechanical/chemical (THM/C) processes in porous media. Environmental Earth Sciences 67(2), 589-599. doi:10.1007/s12665-012-1546-x
○	Land, C.S. (1968). Calculation of imbibition relative permeability for two- and three-phase flow from rock properties. SPE Journal 8(2), 149-156.
✔	Leachman, J.W., Jacobsen, R.T., Penoncello, S.G., Lemmon, E.W. (2009). Fundamental equations of state for parahydrogen, normal hydrogen, and orthohydrogen. J. Phys. Chem. Ref. Data 38, 721-748. doi:10.1063/1.3160306
✔	Lysyy, M., Føyen, T., Johannesen, E.B., Fernø, M., Ersland, G. (2022). Hydrogen relative permeability hysteresis in underground storage. Geophysical Research Letters 49(17). doi:10.1029/2022GL100364
○	McWhorter, D.B., Sunada, D.K. (1990). Exact integral solutions for two-phase flow. Water Resources Research 26(3), 399-413.
✔	Muzny, C.D., Huber, M.L., Kazakov, A.F. (2013). Correlation for the viscosity of normal hydrogen obtained from symbolic regression. J. Chem. Eng. Data 58, 969-979. doi:10.1021/je301273j (erratum 2022, doi:10.1021/acs.jced.2c00523)
✔	Pfeiffer, W.T., Bauer, S. (2015). Subsurface porous media hydrogen storage - scenario development and simulation. Energy Procedia 76, 565-572. doi:10.1016/j.egypro.2015.07.872
✔	Pitz, M., Grunwald, N., Graupner, B., Kurgyis, K., Radeisen, E., Maßmann, J., Ziefle, G., Thiedau, J., Nagel, T. (2023). Benchmarking a new TH2M implementation in OGS-6 with regard to processes relevant for nuclear waste disposal. Environmental Earth Sciences 82, 319. doi:10.1007/s12665-023-10971-7
✔	Ramesh Kumar, K., Honorio, H., Chandra, D., Lesueur, M., Hajibeygi, H. (2023). Comprehensive review of geomechanics of underground hydrogen storage in depleted reservoirs and salt caverns. Journal of Energy Storage 73, 108912. doi:10.1016/j.est.2023.108912
✔	Rubinstein, J.L., Ellsworth, W.L., Dougherty, S.L. (2018). The 2013-2016 induced earthquakes in Harper and Sumner Counties, southern Kansas. Bull. Seismol. Soc. Am. 108(2), 674-689. doi:10.1785/0120170209

	Reference
✔	Sander, R. (2023). Compilation of Henry's law constants (version 5.0.0) for water as solvent. Atmospheric Chemistry and Physics 23, 10901-12440. doi:10.5194/acp-23-10901-2023
○	van Genuchten, M.Th. (1980). A closed-form equation for predicting the hydraulic conductivity of unsaturated soils. Soil Sci. Soc. Am. J. 44, 892-898.
✔	Watney, W.L., Guy, W.J., Byrnes, A.P. (2001). Characterization of the Mississippian chat in south-central Kansas. AAPG Bulletin 85(1), 85-113. doi:10.1306/8626C767-173B-11D7-8645000102C1865D
✔	Kansas Geological Survey (2002). Characterization of Mississippian Osage chat in south-central Kansas (poster series; Aetna Gas Area discovered 1935, ~209.6 Bcf cumulative gas to May 1999). kgs.ku.edu/PRS/Poster/2002/2002-50/
✔	OpenGeoSys documentation and source code, version 6.5.9 (github.com/ufz/ogs, tag 6.5.9; opengeosys.org).

Appendix E. Verification log (what Claude ran)

Check	Result
check_install.py (host and container)	ALL OK
run_benchmarks.py (6 official benchmarks, tag 6.5.9)	6 x PASS
Radial drainage, 3 cycles	580 steps, 0 rejected; RF 0.71 / 0.79 / 0.85; mass balance 1.0 %
Radial imbibition, 3 cycles	645 steps, 20 rejected; RF 0.40 / 0.39 / 0.38; mass balance 0.9 %
Radial 1 cycle in Docker container	RF 0.707 (identical to host)
Radial 1 cycle with gravity	280 steps, 0 rejected; RF 0.708
Fault section	9 372 cells, 187 steps, 0 rejected, 1133 s
sigma - sigma_total vs. p_g - S_L p_c	max difference 0.25 kPa
test_coulomb.py	6 x PASS
compute_h2_properties.py	ideal-gas error 2.2 % (4 MPa) - 11.5 % (20 MPa)
Days 21-25 loops, run exactly as printed (cushion, Land C, 8 sensitivities, dip 45 re-mesh, 3 meshes)	all ran; results in starter/*/expected/days21-25 and day24_dip45
Mesh convergence (1 cycle)	RF 0.855 / 0.707 / 0.699 for 300 / 600 / 2400 cells
Commands in this guide vs. starter scripts	checked by docs/build/check_commands.py

Appendix F. Model properties and their sources

Printed automatically from starter/properties/properties.csv when the guide is built. Tags: **synthetic/literature** = generic or literature-based placeholder; **literature** = taken from a cited source; **computed** = computed by a script (CoolProp). Replace values here only; every model reads this file.

Key	Value	Unit	Tag	Source
T_res	323.15	K	synthetic/literature	Assumed ~25 degC/km gradient + 15 degC surface at ~1.4 km depth; **TO-VERIFY** with bottom-hole temperatures from Barber County logs (KGS)
z_top_res	1450	m	synthetic/literature	Order of magnitude for Mississippian chat in Barber County; **TO-VERIFY** against KGS well tops
p_init	4.0e6	Pa	synthetic/literature	Assumed depleted state (~30% of hydrostatic); replace with field shut-in pressure (**TO-VERIFY**)
S_L_init	0.60	-	synthetic/literature	High water saturation typical of microporous chat (Watney et al. 2001); log-derived value to be swapped in
res_porosity	0.30	-	synthetic/literature	Watney et al. 2001 report 25-50% for reservoir chert facies
res_permeability	1.0e-14	m2	synthetic/literature	Watney et al. 2001: reservoir facies > 5 mD
res_S_Lr	0.45	-	synthetic/literature	Represents microporosity-bound water (Watney et al. 2001 capillary data); **TO-VERIFY**
res_vG_pb	5.0e5	Pa	synthetic/literature	Placeholder; fit to chat mercury-injection data (Watney et al. 2001) **TO-VERIFY**
res_vG_m	0.5	-	synthetic/literature	Placeholder; fit to chat capillary data **TO-VERIFY**
res_S_gr_drain	0.0	-	synthetic/literature	Bracket A: no trapping (upper bound on recovery)
land_C	2.0	-	synthetic/literature	Land (1968); range 1-5 to be tested in sensitivity
res_E	10.0e9	Pa	synthetic/literature	Placeholder generic value; **TO-VERIFY** with sonic/density logs
res_nu	0.25	-	synthetic/literature	Placeholder generic value; **TO-VERIFY**
biot	1.0	-	synthetic/literature	Upper-bound assumption as in OGS TH2M benchmarks; **TO-VERIFY**
rho_solid	2650	kg/m3	synthetic/literature	Density of quartz (standard value)
cap_porosity	0.05	-	synthetic/literature	Generic shale value; **TO-VERIFY**
cap_permeability	1.0e-19	m2	synthetic/literature	Generic shale value; **TO-VERIFY**
cap_vG_pb	2.0e7	Pa	synthetic/literature	Chosen so that seals stay > 99.5% brine-saturated at the uniform initial capillary pressure (capillary equilibrium); **TO-VERIFY** with shale MICP data
cap_E	15.0e9	Pa	synthetic/literature	Generic shale value; **TO-VERIFY**
cap_nu	0.30	-	synthetic/literature	Generic shale value; **TO-VERIFY**
flt_porosity	0.10	-	synthetic/literature	Generic damage-zone value; **TO-VERIFY**
flt_permeability	1.0e-16	m2	synthetic/literature	Sensitivity range 1e-18 to 1e-14
flt_E	5.0e9	Pa	synthetic/literature	Generic; **TO-VERIFY**
flt_nu	0.30	-	synthetic/literature	Generic; **TO-VERIFY**
H2_M	0.00201588	kg/mol	computed	CoolProp (Bell et al. 2014) / Leachman et al. 2009
H2_Tc	33.1443	K	computed	CoolProp 8.0 normal-hydrogen EOS (Leachman et al. 2009)

Key	Value	Unit	Tag	Source
H2_pc	1.29636e6	Pa	computed	CoolProp 8.0 normal-hydrogen EOS (Leachman et al. 2009)
H2_omega	-0.219	-	computed	CoolProp 8.0 normal-hydrogen EOS (Leachman et al. 2009)
H2_viscosity	9.5e-6	Pa s	computed	compute_h2_properties.py; CoolProp with Muzny et al. 2013 correlation
H2_henry	7.8e-6	mol/(m3 Pa)	literature	Sander (2023) compilation; pure water at 298 K - brine salting-out and temperature effect ignored (**TO-VERIFY** at 50 degC and field salinity)
H2_diff_liquid	3.0e-9	m2/s	literature	Value used in the OGS TH2M benchmark bourgeat.prj; **TO-VERIFY**
brine_density	1100	kg/m3	synthetic/literature	Moderate-salinity brine; Batzle & Wang 1992 range; **TO-VERIFY** with Kansas produced-water data
brine_compressibility	4.0e-10	1/Pa	synthetic/literature	Typical brine value (Batzle & Wang 1992 order of magnitude); **TO-VERIFY**
brine_viscosity	6.0e-4	Pa s	synthetic/literature	Pure water at 50 degC is 5.5e-4 Pa s (CoolProp IAPWS-95); salinity raises it; **TO-VERIFY**
rate_inj	0.1	kg/s	synthetic/literature	Screening value; varied in sensitivity
t_cushion	180	day	synthetic/literature	Design choice
t_inj	120	day	synthetic/literature	Seasonal cycle design
t_rest	30	day	synthetic/literature	Seasonal cycle design
t_wd	120	day	synthetic/literature	Seasonal cycle design
n_cycles	3	-	synthetic/literature	Screening (increase to 5-10 for paper)
p_bhp_min	4.0e6	Pa	synthetic/literature	Set equal to p_init so native gas (modelled as H2) is not produced
Sv_grad	23.0e3	Pa/m	synthetic/literature	About 1 psi/ft; **TO-VERIFY** with density log integration
Shmin_ratio	0.65	-	synthetic/literature	Strike-slip/normal regime reported for northern Oklahoma and southern Kansas (Alt & Zoback 2017); value **TO-VERIFY**; sensitivity 0.6-0.8
SHmax_ratio	1.0	-	synthetic/literature	SHmax ~ Sv at the strike-slip/normal transition (Alt & Zoback 2017); **TO-VERIFY**; sensitivity 1.0-1.3
friction	0.6	-	literature	Byerlee (1978); sensitivity 0.4-0.8
fault_dip	60	deg	synthetic/literature	Sensitivity 45-75 deg
r_well	0.1	m	synthetic/literature	Typical production casing scale
re_compartment	500	m	synthetic/literature	Design/sensitivity parameter (250-1000 m); fault-bounded compartment assumed
res_thickness	20	m	synthetic/literature	Placeholder; **TO-VERIFY** from log-derived net pay
cap_S_Lr	0.20	-	synthetic/literature	Generic shale value; **TO-VERIFY**
ob_porosity	0.10	-	synthetic/literature	Generic; **TO-VERIFY**
ob_permeability	1.0e-18	m2	synthetic/literature	Generic; **TO-VERIFY**
ob_E	20.0e9	Pa	synthetic/literature	Generic; **TO-VERIFY**
ob_nu	0.25	-	synthetic/literature	Generic; **TO-VERIFY**
line_rate	1.0e-4	kg/(s m)	synthetic/literature	0.1 kg/s spread over 1 km of horizontal well (or a row of wells); half of it enters the modelled half-section
section_azimuth	0	deg	synthetic/literature	Hypothetical E-W striking fault (strike = section azimuth + 90); **TO-VERIFY** with mapped faults near Aetna

Key	Value	Unit	Tag	Source
SHmax_azimuth	85	deg	literature	N85E +/- 5 deg in Oklahoma (Alt & Zoback 2017); extrapolation to Barber County **TO-VERIFY** (World Stress Map)